

rapidmesh

A Pure-Rust Hierarchical Mesher for B-Rep Solids:
Conforming Tetrahedra, Sliver-Free Surfaces, FEM/MoM Export

Milan Rother

July 4, 2026

Abstract

This document specifies the meshing algorithm rapidmesh uses to turn a boundary representation (B-Rep) of one or more material regions into a conforming, isotropic, high-quality tetrahedral mesh. The algorithm is *dimensionally hierarchical*: it meshes 0-cells (corners), then 1-cells (edges), then 2-cells (faces), then the 3-cell (volume), and *freezes* each level before the next consumes it. Within every dimension it combines *error-driven adaptive sampling* (insert where a geometric tolerance is violated) with *variational point relaxation* for quality—a sizing-weighted centroidal-Voronoi (Lloyd) relaxation on edges and faces, and an *optimal-Delaunay* (ODT) relaxation in the volume. The frozen surface triangulation is a *hard constraint* on the volume Delaunay, which is what makes the boundary watertight by construction. We derive, with a computer-algebra system, the arc-length, curvature, deflection-sizing, first/second fundamental form and distance-faithful chart formulae for every edge and face modality the geometry kernel supports (line, circle/arc, surface-surface intersection, NURBS curve, extruded profile; plane, cylinder, cone, sphere, torus, extruded/ruled, NURBS), and give pseudocode for each stage.

Contents

1	Design philosophy	6
2	Exact booleans: the B-Rep from a region predicate	7
3	Notation and preliminaries	8
3.1	The sizing field	8
3.2	Centroidal Voronoi Tessellation	9
3.3	Discrete centroids and the restricted setting	10
3.4	Optimal-Delaunay relaxation for the volume	10
3.5	Robustness, exactness, and determinism	10
4	Stage 0: Vertices	10
5	Stage 1: Edges	11
5.1	Arc length	11
5.2	Curvature	11
5.3	Intersection curves	11
5.4	Deflection sizing	12
5.5	1D CVT and error-driven adaptivity	12
6	Stage 2: Faces	13
6.1	First fundamental form and area	14
6.2	Curvature and surface deflection sizing	14
6.3	Distance-faithful charts	14
6.4	Surface CVT and adaptivity	18
7	Stage 3: Volume	18
7.1	Sizing field from the surface	18
7.2	Interior seeding	18
7.3	ODT relaxation and quality-driven adaptivity	18
7.4	Boundary-constrained tetrahedralization	19
7.5	Carrier oracles and analytic crossing solvers	20
8	The quality endgame: perturbation and sliver exudation	21
8.1	Degeneracy perturbation	21
8.2	Sliver exudation on weighted Delaunay triangulations	21
9	The sizing field, end to end	22
9.1	Error control: two orthogonal axes	23
10	Complexity and parallel scaling	23
11	Why this guarantees conformity	24
12	Surface meshing and adaptivity	24
12.1	Sliver-free Ruppert/Chew refinement	24
12.2	Element and triangle budgets	24
12.3	Adaptive refinement (Dörfler marking)	25

13 Discrete carriers: remeshing imported triangle soups	25
13.1 Crease segmentation and validation	25
13.2 Curvature estimation and its two governors	25
13.3 Analytic crossings on closed patches	26
13.4 Bridge faces: a fourth defect class	26
14 FEM and MoM export	26
15 Validation	27
15.1 Gallery	31
References	37

List of Figures

1	The bottom-up pipeline. Exact booleans produce the B-Rep (section 2); the mesher then meshes 0-, 1-, 2-, 3-cells in order. Within each of stages 1–3 the entities are independent and run in parallel (the stacked boxes); the only synchronisation is the barrier between levels (Principle 1.4).	7
2	The relax–insert–relax scheme shared by all stages (Principle 1.4, mechanism (i)+(ii)). The error measure e^* is the chord deflection (edges), the triangle–surface deflection (faces) or the element quality (volume); ℓ^* is the longest edge for the $h_{\max}$ bound. Iterate until both the tolerance and $h_{\max}$ hold and the sites stop moving, then freeze.	13
3	Surface meshing through a chart: relax in a (near-)isometric 2D parameterization with the frozen edge points as fixed boundary, then lift every chart point back onto the analytic surface. Exact on developables, bounded distortion on the sphere (equation (6.4)).	16

List of Tables

1	Edge curve modalities. All expose $\mathbf{C}(t), \mathbf{C}'(t), \mathbf{C}''(t)$	11
2	Per-modality geometric quantities (all verified by computer algebra): the parameterization $\mathbf{S}(u, v)$, the first fundamental form E, F, G and area element dA (section 6.1), and the Gaussian curvature K and largest principal curvature $\kappa_{\max} = 1/R_{\min}$ that drive the deflection sizing (section 6.2). $\mathbf{a}$ is the unit axis, α the cone half-angle. All but the NURBS row are diagonal ($F = 0$); developables have $K = 0$; the sphere is umbilic.	15
3	Surface meshes: 2D plates (planar surface stage).	28
4	Volume meshes: Analytic primitives.	28
5	Volume meshes: CSG booleans and multi-region assemblies.	28
6	Volume meshes: Imported reference models (discrete carriers).	29
7	Volume meshes: Showcase models.	29
8	Volume meshes: RF/EM structures.	30
9	Volume meshes: Sizing.	30
10	Surface meshes: Sizing.	30
11	Surface meshes: Shape.	30
12	Volume meshes: Graded.	31
13	Volume meshes: Passives.	31
14	Surface meshes: Passives.	31

1 Design philosophy

Principle 1.1 (Dimensional hierarchy). *A k -cell is meshed only after every $(k-1)$ -cell on its boundary is meshed and frozen. Corners pin edges; edges pin faces; faces pin the volume. Shared lower-dimensional entities are meshed once and reused, so adjacent cells agree on their common boundary exactly (conformity across seams is inherited, not reconstructed).*

Principle 1.2 (Two orthogonal mechanisms per level). *Each dimension uses (i) error-driven adaptivity to decide how many points and where (insert at the worst geometric-error location, then re-relax, until a tolerance is met), and (ii) variational relaxation to decide point quality — a density-weighted centroidal-Voronoi (Lloyd) move on edges and faces, and the optimal-Delaunay (ODT) move (3.8) in the volume. Adaptivity controls density; the relaxation controls regularity.*

Principle 1.3 (Constrained volume, not recovered boundary). *The frozen surface triangulation enters the volume stage as a set of constraint facets. The volume tetrahedralization is required to contain them as faces. The boundary is therefore enforced, not statistically recovered: there are no straddling tetrahedra, hence no “noses” protruding into a void and no gaps. The surface is therefore completed before the volume is seeded.*

Principle 1.4 (Embarrassingly parallel within each level). *Freezing a level before the next consumes it makes the entities within a level mutually independent: they share only already-frozen, read-only lower-dimensional boundary. Hence each level is processed in parallel with no synchronisation:*

- ***all edges** at once (each depends only on its two fixed corners);*
- *then **all faces** at once (each depends only on its fixed boundary edges, which are computed once and read concurrently);*
- *then **all volume regions** at once (each material region’s constrained tetrahedralization depends only on its fixed bounding surface; shared interface facets are read-only).*

Shared lower-dimensional entities are meshed exactly once and cached, which is what removes write contention. Each entity’s result is independently deterministic and merged in fixed id order, so parallelism does not affect reproducibility (section 3). The three barriers (edges $\rightarrow$ faces $\rightarrow$ volume) are the only synchronisation points.

The geometry source is a B-Rep assembled from the exact CSG arrangement: faces are trimmed analytic (or NURBS) surfaces, edges are analytic curves (including the intersection curves booleans create), vertices are corner points, and the radial-edge topology records which faces meet along each edge and the material region on each side. The mesh is built *from this geometry*, independent of any input tessellation. Figure 1 shows the whole pipeline.

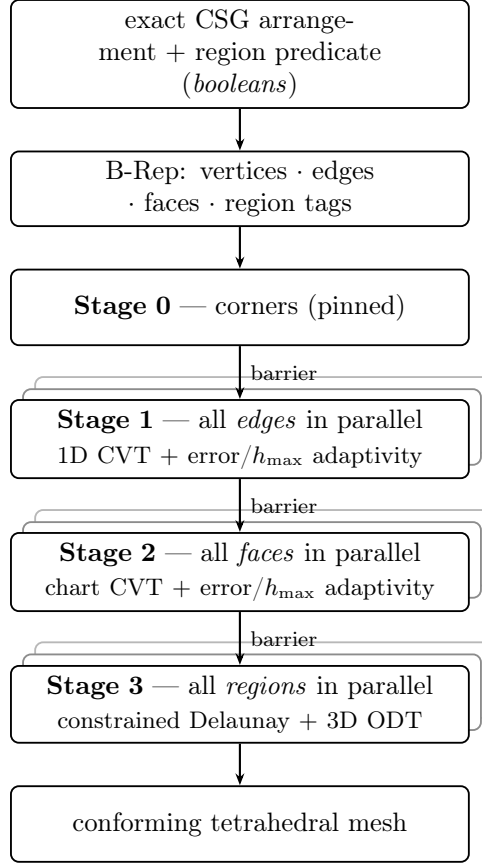


Figure 1: The bottom-up pipeline. Exact booleans produce the B-Rep (section 2); the mesher then meshes 0-, 1-, 2-, 3-cells in order. Within each of stages 1–3 the entities are independent and run in parallel (the stacked boxes); the only synchronisation is the barrier between levels (Principle 1.4).

Algorithm 1: $\text{MESH}(\mathcal{B}, h_{\max}, \varepsilon)$: top-level driver

Input: B-rep $\mathcal{B}$ (corners, edges, faces, region tags); size $h_{\max}$; deflection tolerance ε
Output: conforming, watertight, multi-region tetrahedral mesh

```

pin every corner vertex                                // Stage 0;
foreach edge  $e \in \mathcal{B}$  (in parallel) do
   $P_e \leftarrow \text{MESHEdge}(e, h_{\text{edge}}, \varepsilon)$                                 // Stage 1
barrier;
foreach face  $f \in \mathcal{B}$  (in parallel) do
   $h_f \leftarrow \text{BUILDSIZINGFIELD}(\partial f, \text{curvature}, h_{\max})$ ;  $T_f \leftarrow \text{MESHFACE}(f, h_f, \varepsilon)$ 
  // Stage 2
barrier;
 $\mathcal{S} \leftarrow \bigcup_f T_f$                                 // watertight surface mesh;
 $h_{\text{vol}} \leftarrow \text{BUILDSIZINGFIELD}(\mathcal{S}, \text{surface size}, h_{\max})$ ;
foreach region  $R$  (in parallel) do
   $M_R \leftarrow \text{MESHVOLUME}(\mathcal{S}|_R, h_{\text{vol}})$                                 // Stage 3
return  $\bigcup_R M_R$ , merged in fixed id order;

```

2 Exact booleans: the B-Rep from a region predicate

The mesher’s input is produced upstream by an *exact* boolean stage; no boolean logic lives in the mesher. All operands (closed, outward-oriented analytic or NURBS solids) are arranged

together into one exact arrangement: every pairwise surface intersection is computed with exact predicates [1], splitting the input faces into sub-faces and producing the intersection curves and triple points as exact geometry (rational / implicit points, never snapped to floating point).

Region as a predicate over a containment bit-vector. Each cell of the arrangement (a maximal region crossed by no surface) carries a containment bit-vector $\mathbf{b} \in \{0, 1\}^n$ with $b_k = 1$ iff the cell lies inside operand k , decided *once* by an exact point-in-solid test at the cell’s barycenter. A boolean *operation* is then nothing but a labelling rule, a map $\omega : \{0, 1\}^n \rightarrow$ region tag applied to $\mathbf{b}$:

$$\underbrace{\omega = \bigvee_k b_k}_{\text{union}}, \quad \underbrace{\bigwedge_k b_k}_{\text{intersection}}, \quad \underbrace{b_A \wedge \neg b_B}_{\text{difference}}, \quad \underbrace{b_A \oplus b_B}_{\text{xor}}, \quad \underbrace{\omega(\mathbf{b})}_{\substack{\text{multi-material:} \\ \text{a tag per pattern}}} . \quad (2.1)$$

A sub-face survives iff the region differs across it, $\omega(\mathbf{b}_{\text{front}}) \neq \omega(\mathbf{b}_{\text{back}})$, and that pair labels the resulting B-rep face. Same-region sub-faces (ω equal on both sides) are interior and dropped, which is exactly what *fuses* a union. The background carries the reserved tag 0 (a *void*: not meshed, but its walls survive as boundary). The construction is N -ary and compositional: an arbitrary CSG expression tree evaluates to a single ω , so union, intersection, difference, xor and genuine multi-material assemblies are *one* mechanism, not special cases.

From arrangement to B-Rep. The surviving sub-faces, grouped by (surface, region pair, face tag, connected component), become the B-rep faces; the intersection curves that bound them become the edges (recovered analytically, section 5.3); the points where three or more surfaces meet become the corners. The boolean result therefore enters the mesher as precisely the cell complex that stages 0–3 consume (figure 1), with exact geometry and per-side region labels, and the classification is itself embarrassingly parallel over sub-faces.

Algorithm 2: ASSEMBLEBREP($\{\mathbf{S}_k\}_{k=1}^n, \omega$): exact boolean $\rightarrow$ B-rep

Input: operand solids $\mathbf{S}_1, \dots, \mathbf{S}_n$; region predicate $\omega : \{0, 1\}^n \rightarrow$ tag

Output: B-rep (corners, edges, faces with region pairs)

$\mathcal{A} \leftarrow$ exact arrangement of all $\mathbf{S}_k$ (intersections, sub-faces);

foreach sub-face $\phi \in \mathcal{A}$ (*in parallel*) **do**

$\mathbf{b} \leftarrow$ containment bit-vector at ϕ ’s barycenter // exact point-in-solid;
 $(\mathbf{b}^+, \mathbf{b}^-) \leftarrow \mathbf{b}$ with ϕ ’s own bit set on each side;
 keep ϕ with pair $(\omega(\mathbf{b}^+), \omega(\mathbf{b}^-))$ iff $\omega(\mathbf{b}^+) \neq \omega(\mathbf{b}^-)$;

group kept sub-faces $\rightarrow$ faces; intersection curves $\rightarrow$ edges; triple points $\rightarrow$ corners;

return B-rep;

3 Notation and preliminaries

3.1 The sizing field

A scalar *sizing field* $h : \mathbb{R}^3 \rightarrow \mathbb{R}_{>0}$ prescribes the target element edge length. It is assembled bottom-up (edges set it on 1-cells, faces refine it on 2-cells, the volume smooths it through the interior) and is always made *Lipschitz* (gradation-limited):

$$h(\mathbf{x}) \leq h(\mathbf{y}) + \alpha \|\mathbf{x} - \mathbf{y}\| \quad \forall \mathbf{x}, \mathbf{y}, \quad (3.1)$$

with grading constant $\alpha \in (0, 1]$ (the default $\alpha = \frac{1}{2}$ grows neighbouring elements by at most $\sim 1.5\times$). Equation (3.1) is the standard mesh-gradation constraint [2]. Given raw (ungraded) size requests $h_0(\mathbf{y})$ from the geometry, the largest field that satisfies both $h \leq h_0$ and (3.1) is the lower envelope

$$h(\mathbf{x}) = \min_{\mathbf{y}} (h_0(\mathbf{y}) + \alpha \|\mathbf{x} - \mathbf{y}\|), \quad (3.2)$$

the viscosity solution of the Eikonal inequality $\|\nabla h\| \leq \alpha$ with obstacle $h \leq h_0$. It is computed by a single fast-marching (or a few Gauss–Seidel sweeps) over the size sources, $O(N \log N)$ in the number of grid nodes, the same machinery (algorithm 3) at every stage, fed first by edges, then faces, then the volume.

Algorithm 3: BUILD_SIZING_FIELD: gradation limiting (equation (3.2))

Input: raw requests h_0 at sources (edge spacing, curvature, surface element size);
grading α ; cap $h_{\max}$
Output: Lipschitz field h on the background octree
 $h(\mathbf{x}) \leftarrow \min(h_0(\mathbf{x}), h_{\max})$ at sources, $+\infty$ elsewhere; push sources into a min-heap keyed by h ;
while *heap nonempty* **do**
 pop node $\mathbf{x}$ of smallest h ;
 foreach *neighbour* $\mathbf{y}$ **do**
 if $h(\mathbf{x}) + \alpha \|\mathbf{x} - \mathbf{y}\| < h(\mathbf{y})$ **then**
 $h(\mathbf{y}) \leftarrow h(\mathbf{x}) + \alpha \|\mathbf{x} - \mathbf{y}\|$; update heap
 return h ;

3.2 Centroidal Voronoi Tessellation

Given a density $\rho : \Omega \rightarrow \mathbb{R}_{>0}$ on a domain Ω and sites $\{\mathbf{z}_i\}_{i=1}^n$, the Voronoi region of $\mathbf{z}_i$ is $V_i = \{\mathbf{x} \in \Omega : \|\mathbf{x} - \mathbf{z}_i\| \leq \|\mathbf{x} - \mathbf{z}_j\| \ \forall j\}$. The tessellation is *centroidal* when each site coincides with the mass centroid of its region,

$$\mathbf{z}_i = \mathbf{c}_i := \frac{\int_{V_i} \rho(\mathbf{x}) \mathbf{x} \, d\mathbf{x}}{\int_{V_i} \rho(\mathbf{x}) \, d\mathbf{x}}. \quad (3.3)$$

A CVT is a critical point of the quantization energy

$$\mathcal{F}(\{\mathbf{z}_i\}) = \sum_i \int_{V_i} \rho(\mathbf{x}) \|\mathbf{x} - \mathbf{z}_i\|^2 \, d\mathbf{x} \quad (3.4)$$

[3]. Holding the cells fixed and differentiating with respect to a single site,

$$\frac{\partial \mathcal{F}}{\partial \mathbf{z}_i} = -2 \int_{V_i} \rho(\mathbf{x}) (\mathbf{x} - \mathbf{z}_i) \, d\mathbf{x}, \quad (3.5)$$

where the boundary terms from moving ∂V_i cancel because the integrand $\rho \|\mathbf{x} - \mathbf{z}_i\|^2$ is continuous across the shared Voronoi faces (each face contributes equal and opposite to its two cells). Setting (3.5) to zero gives $\mathbf{z}_i = \int_{V_i} \rho \mathbf{x} / \int_{V_i} \rho = \mathbf{c}_i$, the centroid (3.3). The one-dimensional case is immediate: $\partial_z \int_a^b (x - z)^2 \, dx = 0 \Rightarrow z = \frac{a+b}{2}$. *Lloyd’s algorithm* [4] alternates “recompute V_i ” and “move $\mathbf{z}_i \leftarrow \mathbf{c}_i$ ” and converges to a CVT; the resulting point set is blue-noise / isotropic, which is precisely the input a Delaunay mesher needs for well-shaped elements.

Density that realises a sizing field. To make the local point spacing follow h , choose

$$\rho(\mathbf{x}) = h(\mathbf{x})^{-(d+2)} \quad (\text{theory, } d \text{ dimensions}), \quad \rho(\mathbf{x}) = h(\mathbf{x})^{-d} \quad (\text{used in practice}), \quad (3.6)$$

because a CVT equidistributes the energy density, giving local spacing $\propto \rho^{-1/(d+2)}$; the simpler h^{-d} (number density $\propto$ points per unit volume) is robust and is what we weight cells by. See [5], [6] for the exponent discussion.

3.3 Discrete centroids and the restricted setting

On a triangulation, the integral (3.3) is approximated by summing over the simplices incident to $\mathbf{z}_i$, each weighted by its measure times the density at its centroid. In dimension d with incident simplices T of centroid $\mathbf{g}_T$, measure $|T|$,

$$\mathbf{c}_i \approx \frac{\sum_{T \ni \mathbf{z}_i} |T| \rho(\mathbf{g}_T) \mathbf{g}_T}{\sum_{T \ni \mathbf{z}_i} |T| \rho(\mathbf{g}_T)}. \quad (3.7)$$

On a curved surface, the relevant object is the *restricted* Voronoi diagram (Voronoi cells clipped to the surface) and its dual *restricted Delaunay* triangulation [7], [8]; in practice we relax in a 2D chart (section 6) so (3.7) applies in the chart and the move is lifted back onto the surface.

3.4 Optimal-Delaunay relaxation for the volume

In the volume we relax by the *optimal Delaunay triangulation* (ODT) update rather than the CVT centroid: it minimises the piecewise-linear interpolation error of $\|\mathbf{x}\|^2$ over the triangulation [9], which penalises slivers far more directly than the Voronoi quantisation energy (3.4) and is what the implementation uses. Holding the connectivity fixed the energy is quadratic in one interior site $\mathbf{z}_i$, and its minimiser over the incident tetrahedra $T \ni \mathbf{z}_i$ (measure $|T|$, the three other vertices $\mathbf{v}_1^T, \mathbf{v}_2^T, \mathbf{v}_3^T$) is

$$\mathbf{z}_i^* = \frac{\sum_{T \ni \mathbf{z}_i} |T| (\mathbf{v}_1^T + \mathbf{v}_2^T + \mathbf{v}_3^T)}{3 \sum_{T \ni \mathbf{z}_i} |T|}. \quad (3.8)$$

Sizing enters through the measure weight, exactly as for the surface CVT; the frozen surface vertices never move. The relaxation runs on an *unconstrained* Delaunay rebuilt each pass (approximate cells suffice for the move), and the boundary-constrained build (section 7) is performed *once*, after the interior has settled.

3.5 Robustness, exactness, and determinism

Three properties make the output trustworthy and reproducible. (i) *Exactness*. Every combinatorial decision (orientation, in-circle/in-sphere, point-in-solid) uses exact adaptive-precision predicates [1]; only non-decision quantities (centroid weights, sizing values) are floating point, and the boolean stage never snaps a coordinate. (ii) *Determinism*. The result must not vary between runs, so every hash map whose *iteration order* can influence geometry uses a seedless hash (no per-run random seed), and every reduction across parallel work is merged in a fixed id order. Parallelism (Principle 1.4) therefore does not perturb the mesh: each entity is independently deterministic and the merge order is fixed, so the mesh is bit-identical run to run. (iii) *Robustness*. Where two surfaces meet near-tangentially the boolean can place points within an ulp of each other; the incremental Delaunay *skips* an insertion that would swallow an existing vertex's star rather than corrupting the triangulation, dropping the redundant near-duplicate. The exact predicates guarantee the remaining construction is always a valid triangulation.

4 Stage 0: Vertices

Every B-Rep corner $\mathbf{p}$ becomes a *pinned* site: it never moves and is present in every downstream triangulation. Pinning corners preserves sharp features exactly. Formally these are the 0-cells of the cell complex and the fixed points of all relaxations below.

5 Stage 1: Edges

A B-Rep edge is a curve $\mathbf{C} : [t_0, t_1] \rightarrow \mathbb{R}^3$ between two corners. We support the modalities in table 1.

Table 1: Edge curve modalities. All expose $\mathbf{C}(t)$, $\mathbf{C}'(t)$, $\mathbf{C}''(t)$.

Modality	$\mathbf{C}(t)$	Source
Line	$\mathbf{p}_0 + t \mathbf{d}$	straight B-Rep edge
Circle / arc	$\mathbf{c} + r(\cos t \mathbf{x} + \sin t \mathbf{y})$	rim, fillet, boolean of quadrics
Intersection	implicit $\mathbf{S}_a \cap \mathbf{S}_b$, seeded by the chain	generic surface–surface boolean
NURBS	$\frac{\sum_i N_{i,p}(t) w_i \mathbf{P}_i}{\sum_i N_{i,p}(t) w_i}$	imported / profile curve [10]
Extruded profile	$\mathbf{b} + z \mathbf{a} + u_x P_x(t) + u_y P_y(t)$	lifted 2D profile (airfoil)
Polyline	piecewise linear chain	faceted fallback

5.1 Arc length

The arc length and the natural (unit-speed) parameter are

$$s(t) = \int_{t_0}^t \|\mathbf{C}'(\tau)\| d\tau, \quad ds = \|\mathbf{C}'(t)\| dt. \quad (5.1)$$

We tabulate $s(t)$ once (adaptive Gauss/Simpson) and invert $t(s)$ by table lookup + a Newton step using $dt/ds = 1/\|\mathbf{C}'\|$, so points can be placed at prescribed arc-length spacings. For a circle $\|\mathbf{C}'\| = r$ and $s = r \Delta t$ exactly; for a line it is exact; for NURBS / extruded profiles the table is used.

5.2 Curvature

For a space curve the (unsigned) curvature is

$$\kappa(t) = \frac{\|\mathbf{C}'(t) \times \mathbf{C}''(t)\|}{\|\mathbf{C}'(t)\|^3}, \quad R(t) = \frac{1}{\kappa(t)} \text{ (radius of curvature)}, \quad (5.2)$$

derived from the Frenet apparatus and verified symbolically. A straight segment has $\kappa \equiv 0$, $R \equiv \infty$; a circle of radius r has $\kappa \equiv 1/r$. For a NURBS curve $\mathbf{C} = \mathbf{A}/w$ (homogeneous numerator $\mathbf{A} = \sum_i N_{i,p} w_i \mathbf{P}_i$, weight $w = \sum_i N_{i,p} w_i$) the derivatives follow from the quotient rule,

$$\mathbf{C}' = \frac{\mathbf{A}'w - \mathbf{A}w'}{w^2}, \quad \mathbf{C}'' = \frac{\mathbf{A}'' - 2\mathbf{C}'w' - \mathbf{C}w''}{w}, \quad (5.3)$$

with $\mathbf{A}^{(k)}, w^{(k)}$ from the de Boor recurrence [10]; (5.1)–(5.2) then apply unchanged.

5.3 Intersection curves

A boolean between two surfaces $\mathbf{S}_a, \mathbf{S}_b$ creates an edge along their intersection. Writing the surfaces locally as implicit $f(\mathbf{x}) = 0, g(\mathbf{x}) = 0$ (signed distance to the analytic surface, so ∇f is the unit normal $\mathbf{n}_a$), the curve $\{f = g = 0\}$ has unit tangent

$$\mathbf{T} = \frac{\nabla f \times \nabla g}{\|\nabla f \times \nabla g\|} = \frac{\mathbf{n}_a \times \mathbf{n}_b}{\|\mathbf{n}_a \times \mathbf{n}_b\|}, \quad (5.4)$$

which degenerates only where the surfaces are tangent ($\mathbf{n}_a \parallel \mathbf{n}_b$). A point near the curve is projected *onto both surfaces* simultaneously by a constrained Newton iteration on $\mathbf{F} = (f, g)^\top$ with Jacobian $J = [\nabla f^\top; \nabla g^\top] \in \mathbb{R}^{2 \times 3}$; the minimum-norm step is

$$\mathbf{x} \leftarrow \mathbf{x} - J^+ \mathbf{F}(\mathbf{x}), \quad J^+ = J^\top (J J^\top)^{-1} \quad (2 \times 2 \text{ inverse}), \quad (5.5)$$

quadratically convergent off the tangency locus. The B-rep chain seeds the iteration, and arc length / sizing then use the recovered $\mathbf{T}$ and the curve curvature

$$\kappa = \frac{\|\mathbf{C}' \times \mathbf{C}''\|}{\|\mathbf{C}'\|^3}, \quad \mathbf{C}' \parallel \mathbf{T}, \quad \mathbf{C}'' = (\text{rate of change of } \mathbf{T} \text{ along the curve}), \quad (5.6)$$

obtained by differentiating (5.4) along $\mathbf{T}$ (or, where both surfaces are analytic with known second fundamental forms, from their normal curvatures in the direction $\mathbf{T}$). A circle is recovered exactly when both operands are quadrics sharing an axis, in which case the modality collapses to the *Circle* row of table 1.

Algorithm 4: INTERSECTIONCURVE($\mathbf{S}_a, \mathbf{S}_b, \text{chain}$)

Input: two surfaces (implicit f, g); seed polyline (chain) from the arrangement

Output: points on $\{f=g=0\}$ with tangents, ready for MESHEDGE

foreach seed vertex $\mathbf{x}_0$ of the chain **do**

repeat
 $\mathbf{x} \leftarrow \mathbf{x} - J^+(f, g)^\top, \quad J = [\nabla f^\top; \nabla g^\top] \quad // \text{ (5.5)}$
 until $\|(f, g)\| < \text{tol}$;
 $\mathbf{T} \leftarrow (\nabla f \times \nabla g) / \|\nabla f \times \nabla g\| \quad // \text{ (5.4)}$

distribute along the recovered curve by deflection (5.8), using κ from (5.6);

5.4 Deflection sizing

The chord between two samples a parametric distance apart deviates from the true arc by the *sagitta*. For an arc of radius R subtending a half-angle $\theta/2$, with chord length $\ell = 2R \sin(\theta/2)$, the maximum chord–arc deviation is

$$\varepsilon = R(1 - \cos \frac{\theta}{2}) = \frac{R \theta^2}{8} + O(\theta^4) \approx \frac{\ell^2}{8R}. \quad (5.7)$$

Solving the small-angle relation for the admissible edge length given a *deflection tolerance* ε gives the local target

$$\boxed{\ell(\mathbf{x}) = \sqrt{8\varepsilon R(\mathbf{x})}} \quad \left(\text{exact: } \ell = 2\sqrt{R^2 - (R - \varepsilon)^2} \right), \quad (5.8)$$

both verified with a computer-algebra system. The edge sizing field is then

$$h_{\text{edge}}(\mathbf{x}) = \min\left(h_{\text{max}}, \sqrt{8\varepsilon R(\mathbf{x})}\right), \quad (5.9)$$

i.e. refine where the curve bends (R small), coast at h_{max} where it is straight. This is the curve analogue of the surface bound (6.3).

5.5 1D CVT and error-driven adaptivity

On a fixed edge the n interior sites $s_1 < \dots < s_n$ (arc-length coordinates, endpoints pinned) relax by the 1D centroid rule (the segment analogue of (3.7)): with neighbours s_{i-1}, s_{i+1} and density $\rho = h^{-1}$,

$$s_i \leftarrow \frac{\rho_{i-\frac{1}{2}} \frac{s_{i-1} + s_i}{2} + \rho_{i+\frac{1}{2}} \frac{s_i + s_{i+1}}{2}}{\rho_{i-\frac{1}{2}} + \rho_{i+\frac{1}{2}}}, \quad \rho_{i \pm \frac{1}{2}} = h\left(\frac{s_i + s_{i \pm 1}}{2}\right)^{-1}, \quad (5.10)$$

which equidistributes spacing $\propto h$. Adaptivity wraps the relaxation (algorithm 5): relax for quality, then split wherever the chord deflection exceeds ε or a segment exceeds $h_{\max}$. All three meshing stages share this single relax–insert loop (figure 2); only the error measure and the centroid rule change with the dimension.

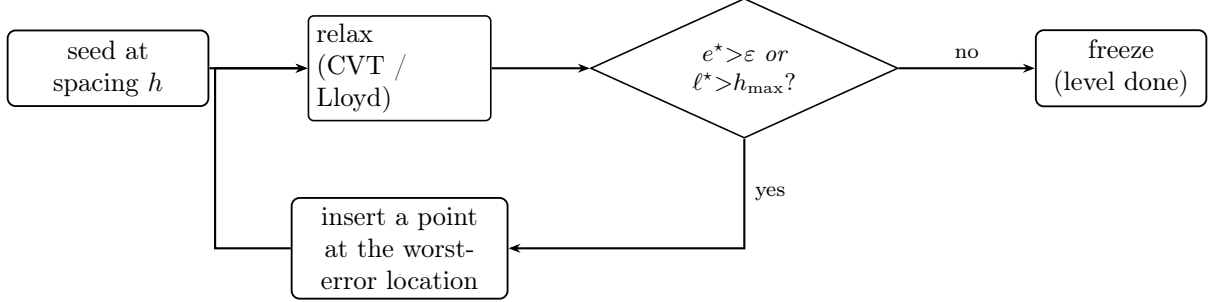


Figure 2: The relax–insert–relax scheme shared by all stages (Principle 1.4, mechanism (i)+(ii)). The error measure e^* is the chord deflection (edges), the triangle–surface deflection (faces) or the element quality (volume); ℓ^* is the longest edge for the $h_{\max}$ bound. Iterate until both the tolerance and $h_{\max}$ hold and the sites stop moving, then freeze.

Algorithm 5: MESHEDGE($\mathbf{C}, h, \varepsilon$)

Input: edge curve $\mathbf{C}$; sizing field h ; deflection tolerance ε

Output: frozen edge points (corners included)

tabulate $s(t)$; $L \leftarrow s(t_1)$;

$n \leftarrow \max(1, \lceil \int_0^L h(s)^{-1} ds \rceil)$; // initial count from the sizing field

place $s_1, \dots, s_n$ equidistributing h (cumulative- ρ inversion);

repeat

for a few iterations do

 relax all s_i by (5.10);

$e^*, s^* \leftarrow$ worst chord–arc deflection and its location; // error-driven

$\ell^*, m^* \leftarrow$ longest segment length and its midpoint; // $h_{\max}$ -driven

if $e^* > \varepsilon$ **or** $\ell^* > h_{\max}$ **then**

 insert a site (at s^* if $e^* > \varepsilon$, else m^*); $n += 1$;

until $e^* \leq \varepsilon$ **and** $\ell^* \leq h_{\max}$ **and** $\max$ site move $< \tau_{\text{move}}$;

return $\{\mathbf{C}(s_i)\}$;

Shared edges are meshed once and cached, so both incident faces read identical points, and all edges run in parallel (Principle 1.4).

6 Stage 2: Faces

A face is a trimmed surface $\mathbf{S} : \Omega \subset \mathbb{R}^2 \rightarrow \mathbb{R}^3$, $(u, v) \mapsto \mathbf{S}(u, v)$, bounded by oriented loops of frozen edges. The strategy is: parameterise the face by a *distance-faithful chart*, relax interior points there with a 2D surface-CVT (boundary = frozen edge points, held fixed), insert where the surface deflection tolerance is violated, and lift every chart point exactly onto $\mathbf{S}$. The output is a *conforming triangulation of the face* that the volume stage will treat as a hard constraint.

Face categories and the face sizing field. Three chart families cover all modalities (section 6.3): *planar* faces use the plane chart; *developable* analytic faces (cylinder, cone, extruded) use an exact isometric unroll; *doubly-curved* analytic faces (sphere, torus) and *periodic* full revolutions use a distance-faithful chart or the periodic revolution construction, with a restricted-CVT fallback for general NURBS. Before any face is seeded, its sizing field is assembled from the

now-frozen edge sampling on its boundary (the edge spacings and any per-region $h_{\max}$) combined with the surface deflection bound (6.3), then Lipschitz-smoothed (3.1) across the face. The frozen edges therefore both bound the face and seed its density.

6.1 First fundamental form and area

With $\mathbf{S}_u = \partial \mathbf{S} / \partial u$, $\mathbf{S}_v = \partial \mathbf{S} / \partial v$, the metric (first fundamental form) coefficients and area element are

$$E = \mathbf{S}_u \cdot \mathbf{S}_u, \quad F = \mathbf{S}_u \cdot \mathbf{S}_v, \quad G = \mathbf{S}_v \cdot \mathbf{S}_v, \quad dA = \sqrt{EG - F^2} \, du \, dv. \quad (6.1)$$

A curve $t \mapsto (u(t), v(t))$ in parameter space has length $\int \sqrt{E\dot{u}^2 + 2F\dot{u}\dot{v} + G\dot{v}^2} \, dt$; this is the metric the chart must reproduce for Euclidean 2D operations (separation, centroid) to mean arc length on the surface. Table 2 lists E, F, G, dA for every analytic modality, all derived with a computer-algebra system.

Worked example (sphere). For $\mathbf{S}(\lambda, \varphi) = r(\cos \varphi \cos \lambda, \cos \varphi \sin \lambda, \sin \varphi)$,

$$\mathbf{S}_u = r(-\cos \varphi \sin \lambda, \cos \varphi \cos \lambda, 0), \quad \mathbf{S}_v = r(-\sin \varphi \cos \lambda, -\sin \varphi \sin \lambda, \cos \varphi),$$

so $E = \mathbf{S}_u \cdot \mathbf{S}_u = r^2 \cos^2 \varphi$, $F = \mathbf{S}_u \cdot \mathbf{S}_v = 0$, $G = \mathbf{S}_v \cdot \mathbf{S}_v = r^2$, giving $dA = r^2 |\cos \varphi| \, d\lambda \, d\varphi$ (the $\cos \varphi$ is the polar foreshortening). With outward normal $\mathbf{n} = \mathbf{S}/r$ the second fundamental form is $L = \mathbf{S}_{\lambda\lambda} \cdot \mathbf{n} = -r \cos^2 \varphi$, $M = \mathbf{S}_{\lambda\varphi} \cdot \mathbf{n} = 0$, $N = \mathbf{S}_{\varphi\varphi} \cdot \mathbf{n} = -r$, hence the principal curvatures $\kappa_1 = L/E = -1/r$ and $\kappa_2 = N/G = -1/r$ are equal (the sphere is umbilic) and $K = \kappa_1 \kappa_2 = 1/r^2$, the table 2 sphere row. The other rows follow the same computation.

6.2 Curvature and surface deflection sizing

The shape operator's eigenvalues are the principal curvatures κ_1, κ_2 ; from the second fundamental form $L = \mathbf{S}_{uu} \cdot \mathbf{n}$, $M = \mathbf{S}_{uv} \cdot \mathbf{n}$, $N = \mathbf{S}_{vv} \cdot \mathbf{n}$ ($\mathbf{n} = \mathbf{S}_u \times \mathbf{S}_v / \|\mathbf{S}_u \times \mathbf{S}_v\|$), the Gaussian and mean curvatures are

$$K = \kappa_1 \kappa_2 = \frac{LN - M^2}{EG - F^2}, \quad H = \frac{1}{2}(\kappa_1 + \kappa_2) = \frac{EN - 2FM + GL}{2(EG - F^2)}, \quad (6.2)$$

and $\kappa_{1,2} = H \pm \sqrt{H^2 - K}$. The per-modality values in table 2 confirm the geometric intuition: developables have $K = 0$ (cylinder, cone, extruded), and the sphere is umbilic ($\kappa_1 = \kappa_2 = 1/r$). For the cone the single non-zero principal curvature is $\cos \alpha / (s \tan \alpha)$, growing without bound toward the apex; for the torus the two principal curvatures are $1/r$ (tube) and $\cos \phi / (R + r \cos \phi)$ (which changes sign across the inner/outer equator), so K is signed.

The surface sagitta argument mirrors (5.7) with $R = R_{\min} = 1/\kappa_{\max}$: a triangle edge of length ℓ on a surface of smallest principal radius $R_{\min}$ deviates from the surface by $\varepsilon \approx \ell^2 / (8R_{\min})$, so the face sizing field is

$$h_{\text{face}}(\mathbf{x}) = \min\left(h_{\max}, \sqrt{8\varepsilon R_{\min}(\mathbf{x})}\right), \quad R_{\min} = 1/\kappa_{\max}. \quad (6.3)$$

6.3 Distance-faithful charts

A chart $\Phi : \text{face} \rightarrow \mathbb{R}^2$ lets the planar CVT machinery act on the surface (figure 3). We want Φ as close to an *isometry* as possible so Euclidean 2D distance equals surface arc length.

Table 2: Per-modality geometric quantities (all verified by computer algebra): the parameterization $\mathbf{S}(u, v)$, the first fundamental form E, F, G and area element dA (section 6.1), and the Gaussian curvature K and largest principal curvature $\kappa_{\max} = 1/R_{\min}$ that drive the deflection sizing (section 6.2). $\mathbf{a}$ is the unit axis, α the cone half-angle. All but the NURBS row are diagonal ($F = 0$); developables have $K = 0$; the sphere is umbilic.

Surface	$\mathbf{S}(u, v)$	E	F	G	dA	K	$\kappa_{\max} = 1/R_{\min}$
Plane	$\mathbf{o} + u\mathbf{e}_1 + v\mathbf{e}_2$	1	0	1	$du dv$	0	0
Cylinder (θ, h)	$r(\cos \theta, \sin \theta) + h\mathbf{a}$	r^2	0	1	$r du dv$	0	$1/r$
Sphere (λ, φ)	$r(\cos \varphi \cos \lambda, \cos \varphi \sin \lambda, \sin \varphi)$	$r^2 \cos^2 \varphi$	0	r^2	$r^2 \cos \varphi du dv$	$1/r^2$	$1/r$
Cone (θ, s)	$s \tan \alpha (\cos \theta, \sin \theta) + s\mathbf{a}$	$s^2 \tan^2 \alpha$	0	$\sec^2 \alpha$	$ s \tan \alpha \sec \alpha du dv$	0	$\frac{1/r}{\cos \alpha}$
Torus (θ, ϕ)	$(R+r \cos \phi)(\cos \theta, \sin \theta) + r \sin \phi \mathbf{a}$	$(R+r \cos \phi)^2$	0	r^2	$r R+r \cos \phi du dv$	$\frac{\cos \phi}{r(R+r \cos \phi)}$	$1/r$
Extruded (t, h)	$\mathbf{b} + P(t) + h\mathbf{a}$	$\ P'(t)\ ^2$	0	1	$\ P'(t)\ du dv$	0	$\frac{ P' \times P'' }{\ P'\ ^3}$
NURBS (u, v)	tensor B-spline [10]	$\mathbf{S}_u \cdot \mathbf{S}_u$	$\mathbf{S}_u \cdot \mathbf{S}_v$	$\mathbf{S}_v \cdot \mathbf{S}_v$	$\sqrt{EG-F^2}$	$\frac{LN-M^2}{EG-F^2}$	$H + \sqrt{H^2 - K}$

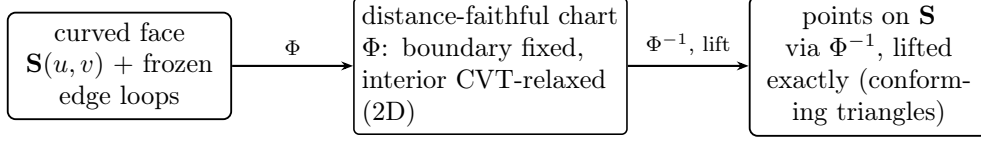


Figure 3: Surface meshing through a chart: relax in a (near-)isometric 2D parameterization with the frozen edge points as fixed boundary, then lift every chart point back onto the analytic surface. Exact on developables, bounded distortion on the sphere (equation (6.4)).

Developables (cylinder, cone, extruded): exact isometry. A surface with $K \equiv 0$ unrolls into the plane without distortion.

- *Cylinder:* $\Phi(\theta, h) = (r\theta, h)$. Then $E = G = 1, F = 0$ in the chart, exact.
- *Cone:* polar unroll $\Phi(\theta, s) = (\rho \cos \psi, \rho \sin \psi)$ with slant $\rho = s / \cos \alpha$ and sweep $\psi = \theta \sin \alpha$. The induced chart metric is $E = s^2 \tan^2 \alpha$, $F = 0$, $G = \sec^2 \alpha$, identical to the cone's first fundamental form (table 2), i.e. *exactly isometric*.
- *Extruded:* $\Phi(t, h) = (\sigma(t), h)$ with $\sigma(t) = \int^t \|P'\|$ the profile arc length; $E = G = 1, F = 0$, exact.

Sphere: azimuthal-equidistant. The sphere ($K = 1/r^2 \neq 0$) admits no isometry to the plane. We use the azimuthal-equidistant chart about a cap centre: the chart radial coordinate is the exact geodesic distance $r\psi$ (ψ the geodesic colatitude), so radial distances are exact; the circumferential direction stretches by

$$\text{stretch}(\psi) = \frac{\psi}{\sin \psi} = 1 + \frac{\psi^2}{6} + \frac{7\psi^4}{360} + O(\psi^6), \quad (6.4)$$

which is ≤ 1.05 for caps up to $\psi \approx 32^\circ$ and stays bounded well past a hemisphere. The chart is a bijection on any cap that excludes the antipode; bijectivity is validated by a round-trip test $\text{dist}(\text{proj}_{\mathbf{S}}(\mathbf{p}), \Phi^{-1}\Phi(\mathbf{p})) < \text{tol}$.

Forward and inverse maps. Each chart provides Φ (surface point $\rightarrow$ chart, to place the fixed boundary) and Φ^{-1} (chart $\rightarrow$ surface, to lift relaxed interior points):

$$\begin{aligned}
 \text{cylinder : } \quad & \Phi(\theta, h) = (r\theta, h), & \Phi^{-1}(x, y) &= \mathbf{S}(x/r, y); \\
 \text{cone : } \quad & \Phi(\theta, s) = \frac{s}{\cos \alpha}(\cos \psi, \sin \psi), \quad \psi = \theta \sin \alpha, & \Phi^{-1}(x, y) &= \mathbf{S}\left(\frac{\text{atan2}(y, x)}{\sin \alpha}, \cos \alpha \sqrt{x^2 + y^2}\right); \\
 & & & (6.5) \\
 \text{extruded : } \quad & \Phi(t, h) = (\sigma(t), h), & \Phi^{-1}(x, y) &= \mathbf{S}(\sigma^{-1}(x), y); \\
 \text{sphere : } \quad & \Phi = \log_{\mathbf{c}}, \quad \Phi^{-1} = \exp_{\mathbf{c}} & & (\text{geodesic polar about } \mathbf{c}).
 \end{aligned}$$

For the sphere, $\log_{\mathbf{c}}(\mathbf{p})$ returns the chart point at radius $r\psi$ ($\psi = \arccos(\hat{\mathbf{c}} \cdot \hat{\mathbf{p}})$ the geodesic colatitude) along the bearing of the tangent toward $\mathbf{p}$; $\exp_{\mathbf{c}}(\mathbf{u})$ walks the geodesic of length $\|\mathbf{u}\|$ from $\mathbf{c}$. The extruded σ^{-1} is the arc-length table inversion of section 5.1. Every lift lands *exactly* on the analytic surface, so the resulting boundary vertices satisfy the same on-surface guarantee as an explicit closest-point projection.

General NURBS / non-developable: metric-scaled or restricted CVT. For a NURBS patch we scale the parameters by the local metric ($\sqrt{E}, \sqrt{G}$) at the patch centroid (a first-order isometry) and, when a single chart distorts too much, fall back to a genuine restricted-Voronoi relaxation on the surface [6], [11]: the centroid (3.7) is computed from the restricted cells and the move is projected back onto $\mathbf{S}$ by closest-point.

Revolution faces and periodicity. A full revolution (cylinder/cone/torus barrel) is periodic in θ ; its boundary loops are rim circles that do not close in the unrolled rectangle. Such a face is meshed by a structured (θ, v) grid whose rings reuse the *rim* longitudes (radial alignment, no seam) and whose row spacing follows h in arc length; the uniform grid is the on-surface CVT optimum of a developable. A sphere cap meeting an intersection circle is itself a revolution about the line of centres (the circle lies in a plane $\perp$ that axis), so the same latitude/longitude construction applies, with both incident caps deriving their longitude reference from the *shared* rim so their rings align.

Algorithm 6: BUILDCHART($\mathbf{S}$, *verts*)

Input: analytic surface $\mathbf{S}$; representative on-surface points of the face
Output: chart (Φ, Φ^{-1}) , or NONE (caller falls back)
switch *kind of* $\mathbf{S}$ **do**
 case *plane* **do**
 | $(\Phi, \Phi^{-1}) \leftarrow$ in-plane orthonormal frame
 case *cylinder / cone / extruded* **do**
 | $(\Phi, \Phi^{-1}) \leftarrow$ exact isometric unroll (6.5)
 case *sphere* **do**
 | $\mathbf{c} \leftarrow$ projected centroid of *verts*; $(\Phi, \Phi^{-1}) \leftarrow (\log_{\mathbf{c}}, \exp_{\mathbf{c}})$
 otherwise do
 | metric-scale the parameters by $(\sqrt{E}, \sqrt{G})$ at the centroid
if $\max_{\text{verts}} \text{dist}(\text{proj}_{\mathbf{S}}(\mathbf{p}), \Phi^{-1}\Phi(\mathbf{p})) > \text{tol}$ **then**
 | **return** NONE // wrapping/closed $\rightarrow$ revolution grid
return (Φ, Φ^{-1}) ;

Algorithm 7: REVOLUTIONGRID($\mathbf{S}$, $\mathbf{a}$, rim loops, h)

Input: revolution surface $\mathbf{S}$; axis $\mathbf{a}$; rim edge points; field h
Output: interior surface points (seam-free, rim-aligned)
 $\{\theta_k\} \leftarrow$ longitudes of the rim points about $\mathbf{a}$ (sorted, dedup) // shared by both
 incident faces;
 $[v_0, v_1] \leftarrow$ the v -extent of the rim loops;
 $n_v \leftarrow \max(1, \lceil \text{arclen}(v_0, v_1)/h \rceil)$;
for interior row $j = 1, \dots, n_v - 1$ **do**
 | $v_j \leftarrow v_0 + (v_1 - v_0)j/n_v$;
 | **for** each θ_k (thinned by $\cos$ toward a pole) **do**
 | emit $\mathbf{S}(\theta_k, v_j)$ // ring aligned to the rim
return the interior rings // rims/poles are the shared edges;

6.4 Surface CVT and adaptivity

Algorithm 8: MESHFACE($\mathbf{S}$, loops, h , ε)

Input: trimmed surface $\mathbf{S}$; frozen boundary loops; field h ; tolerance ε
Output: conforming triangulation of the face (triangles + on-surface vertices)
 build chart Φ (modality-driven); validate bijectivity;
 $B \leftarrow \Phi(\text{frozen edge points of all loops})$; // fixed boundary in chart
 scatter interior sites in $\Phi(\text{face})$ at spacing h (greedy graded Poisson) inside the trim loops;
repeat
 for a few iterations **do**
 2D Delaunay of $B \cup \{\text{interior}\}$ (exact predicates);
 move each interior site by (3.7), $\rho = h^{-2}$, clamp inside the loops; keep B fixed;
 $e^* \leftarrow \max \text{triangle deflection vs. } \mathbf{S} \text{ (sagitta, (6.3))}$; // error-driven
 $\ell^* \leftarrow \max \text{triangle edge length (arc length)}$; // $h_{\max}$ -driven
 if $e^* > \varepsilon$ **or** $\ell^* > h_{\max}$ **then**
 insert a site at the offending triangle's circumcentre (in the chart);
 until $e^* \leq \varepsilon$ **and** $\ell^* \leq h_{\max}$ **and** $\max \text{move} < \tau_{\text{move}}$;
 lift $\mathbf{p}_k = \mathbf{S}(\Phi^{-1}(\text{chart site}_k))$, exact on surface;
return triangulation;

After Stage 2 every face carries a frozen, conforming triangulation; shared edges guarantee face-to-face conformity, and all faces run in parallel (Principle 1.4). The union of all face triangulations is a watertight surface mesh $\mathcal{S}$.

7 Stage 3: Volume

7.1 Sizing field from the surface

The volume field is the same gradient-limited sizing field ((3.1)) carried into the interior, with the *final* surface vertices as its sources — each contributing its local surface element size — so it already reflects the curvature- and feature-driven refinement of the completed surface. It is capped per region by the region's own `maxh` and by the global `maxh_vol`, and is queried from an octree of the surface points so distant sources cost nothing. A region whose walls come close (a thin web, a slender bore wall) is therefore resolved wherever the surface there was already refined.

7.2 Interior seeding

Interior sites are placed on a graded lattice at the local field density: a regular grid at step $\approx h$ is walked, and a candidate is kept only if it lies inside the region, is at least one local element clear of the surface (a closer seed would force a tetrahedron the constrained build cannot emit), and is not within $0.6h$ of an already-accepted site (a spatial-hash separation guard). Surface vertices are *not* re-seeded; they are the fixed boundary of the volume point set.

7.3 ODT relaxation and quality-driven adaptivity

Interior sites relax by the optimal-Delaunay update (3.8) on an *unconstrained* Delaunay rebuilt each pass; the surface vertices stay fixed, and a relocation that would leave the domain or come within a local element of the surface is rejected (the site keeps its position). After a pass, sites are inserted at the longest edge of any tetrahedron coarser than the field, and the relaxation continues. The loop terminates adaptively — once the layout has settled (maximum move below

a fraction of the spacing) *and* seeding has stopped, or once the in-domain sliver count has plateaued — under a hard pass cap; the boundary-constrained build below then runs once.

7.4 Boundary-constrained tetrahedralization

The volume is the *constrained Delaunay tetrahedralization* (CDT) of the interior sites *plus* the frozen surface mesh $\mathcal{S}$ as constraint facets [12], [13], [14]:

$$\text{Tetrahedralize}(\{\text{interior sites}\} \cup V(\mathcal{S})) \quad \text{s.t. every } \Delta \in \mathcal{S} \text{ is a union of tet faces.} \quad (7.1)$$

Because $\mathcal{S}$ is watertight and every constraint triangle is recovered as tetrahedron faces, each surface triangle has a tetrahedron exactly on each side; region labels are then assigned by flood fill across non-constraint faces starting from the region tag attached to each surface triangle’s two sides. No tetrahedron straddles the boundary, so:

Per-region parallelism. Each material region is bounded by a disjoint subset of the frozen $\mathcal{S}$ (its outer walls and the interface facets it shares with neighbours, all read-only), so the regions’ constrained tetrahedralizations are independent and run in parallel (Principle 1.4); a shared interface facet is meshed once on $\mathcal{S}$ and both regions tetrahedralize against the same fixed triangles, which keeps the multi-material mesh conforming across interfaces.

Proposition 7.1 (Watertightness). *If $\mathcal{S}$ is a closed, non-self-intersecting, conforming surface mesh and the CDT recovers it, then $\mathcal{S}$ is covered by tet faces with exactly one tetrahedron on each side (a planar facet may be re-triangulated within its own plane, which leaves the surface geometry unchanged), each side carrying the region label prescribed by $\mathcal{S}$, and the boundary of each meshed region is exactly the corresponding subset of $\mathcal{S}$. In particular there are no protrusions into a void and no gaps.*

The constraint is honoured *without* a separate facet-recovery search. A surface vertex is inserted exactly on its carrier (a point on a line, $\mathbf{x} = \mathbf{a} + t(\mathbf{b} - \mathbf{a})$, or on a plane, $\mathbf{x} = \mathbf{a} + u(\mathbf{b} - \mathbf{a}) + v(\mathbf{c} - \mathbf{a})$, so it stays exactly on the carrier under any later split). *Planar facets then need no recovery:* their vertices are coplanar, so the Delaunay restricted to the plane already tiles them with a tetrahedron on each side, leaving a planar region’s volume bit-exact. Curved facets are *not* force-recovered—a chosen curved triangulation is in general far from Delaunay, so forcing it would need unbounded Steiner insertion, and the chord is a tolerance approximation anyway (section 9.1). Instead a multi-material scene is decomposed *per region* (algorithm 9): each region is tetrahedralized independently against its own subset of the frozen surface, and every tetrahedron whose centroid does not classify into that region by the exact inside predicate is discarded, so no tetrahedron straddles a region interface; the shared interface vertices (one global index per vertex) keep adjacent regions conforming. A single-region body is the constrained Delaunay of its frozen surface directly. The curved boundary of a region is thus the restricted Delaunay of its frozen-surface points, watertight by proposition 7.1 and free of straddlers by construction.

Algorithm 9: MESHREGION($\mathcal{S}, r, \text{interior}$): per-region constrained fill (straddler-free by construction)

Input: frozen surface $\mathcal{S}$ (region-tagged); region r ; interior sites

Output: tetrahedra of region r , conforming on shared interfaces

$F_r \leftarrow \{\Delta \in \mathcal{S} : r \in \text{regions}(\Delta)\}$ // outer walls + interfaces bounding r ;

$V_r \leftarrow \text{vertices of } F_r \text{ (exact on carriers)} \cup \{p \in \text{interior} : \text{region}(p) = r\}$;

$\mathcal{T}_r \leftarrow \text{constrained Delaunay of } V_r \text{ with } F_r \text{ as planar constraints};$

foreach $t \in \mathcal{T}_r$ **do if** region(centroid t) $\neq r$ **then** discard t ;

;

return the kept tetrahedra, tagged r // interface verts shared by global index
 $\Rightarrow$ conforming;

Quality measures and the cleanup pass. Element quality is reported as the minimum dihedral angle and the *radius-edge ratio* $\rho = R_{\text{circ}}/\ell_{\text{min}}$; Delaunay refinement bounds ρ , but *slivers* (near-zero volume, dihedral $\rightarrow 0$, yet acceptable ρ) can survive and are the residual enemy. They are removed by a conformity-preserving post-pass that leaves $\mathcal{S}$ and the region partition intact: (i) smoothing each free vertex toward its quality-weighted optimum, boundary vertices only *within their surface/edge carrier* (a tangential move that, to first order, leaves each region’s enclosed volume unchanged and keeps interface vertices exactly on their analytic carrier); (ii) topological operations — 2–3 face flips and an edge removal that re-triangulates the k -tetrahedron ring around an interior edge by the dynamic program maximising its worst dihedral; and (iii) a dedicated sliver stage that, for an interior sliver, inserts a vertex whose position a pattern search drives to maximise the worst cone dihedral (a local perturbation), and, for a boundary sliver whose vertices are all pinned, splits the offending surface face 1–3 at a Steiner point *on* the analytic carrier (so proposition 7.1 is preserved). Each accepted operation strictly improves the local worst dihedral [9], [15]. This monotone regime converges, but it converges *into a wall*: the surviving sliver population sits in configurations every quality-monotone operator rejects. Lifting that residue is the job of a dedicated endgame (section 8) built on *weighted* Delaunay machinery rather than on quality gates.

7.5 Carrier oracles and analytic crossing solvers

Everything the refinement loop asks of a curved carrier goes through a three-call oracle: the closest point $\Pi(\mathbf{x})$ on the carrier, the *signed offset*

$$\psi(\mathbf{x}) = \langle \mathbf{x} - \Pi(\mathbf{x}), \mathbf{n}(\Pi(\mathbf{x})) \rangle, \quad (7.2)$$

whose sign flips exactly at the surface, and the *crossing set* $X(\mathbf{a}, \mathbf{b}) \subset (0, 1)$ of parameters where the segment $\mathbf{a} \rightarrow \mathbf{b}$ crosses the carrier transversally. The generic crossing solver is a *sphere trace*: because $|\psi|$ lower-bounds the true distance of a 1-Lipschitz signed-distance-like field, marching with the step

$$\Delta t = \max\left(\frac{0.9|\psi|}{\|\mathbf{b} - \mathbf{a}\|}, \Delta t_{\text{min}}\right) \quad (7.3)$$

cannot skip a crossing, and each bracketed sign change is bisected. That is robust but expensive — each evaluation of (7.2) is a projection — so the frequent carriers get *closed-form* solvers. For a plane through $\mathbf{p}_0$ with normal $\mathbf{n}$ the offset is linear in t and the single root is

$$t = \frac{s_0}{s_0 - s_1}, \quad s_i = \langle \mathbf{x}_i - \mathbf{p}_0, \mathbf{n} \rangle. \quad (7.4)$$

For a sphere (centre $\mathbf{c}$, radius r) and a cylinder (axis-orthogonal components $\mathbf{u}_\perp$), the crossings are the roots in $(0, 1)$ of

$$At^2 + 2Bt + C = 0, \quad \begin{aligned} A &= \langle \mathbf{d}, \mathbf{d} \rangle, & B &= \langle \mathbf{w}, \mathbf{d} \rangle, & C &= \langle \mathbf{w}, \mathbf{w} \rangle - r^2, \\ \mathbf{d} &= \mathbf{b} - \mathbf{a}, & \mathbf{w} &= \mathbf{a} - \mathbf{c}, \end{aligned} \quad (7.5)$$

evaluated in the cancellation-safe Citardauq pairing $t_1 = q/A$, $t_2 = C/q$ with $q = -(B + \text{sign}(B)\sqrt{B^2 - AC})$; a tangent root (zero discriminant) is a graze, not a crossing, and is skipped. Piecewise carriers (tube paths, and the discrete patches of section 13) generalise this to a *candidate-verify* pattern: cheap per-piece candidate roots, then one pass of (7.2) sign probes over the candidate partition keeps exactly the transversal subset. Endpoint semantics are asymmetric by construction: a segment starting exactly on the carrier ($\psi = 0$) contributes no crossing at $t = 0$, and a root within float noise (64ε of the coordinate magnitude) of the far endpoint is dropped — matching the traced semantics that the classification walks downstream are calibrated against. Replacing traces by (7.4)–(7.5) removed $\sim 90\text{M}$ oracle projections from a single coil mesh.

Algorithm 10: MESHVOLUME($\mathcal{S}, h$)

Input: watertight surface mesh $\mathcal{S}$ (with region tags); field h
Output: tetrahedra, per-tet region, boundary faces ($= \mathcal{S}$)
build h_{vol} from $\mathcal{S}$ (surface sources, gradation-limit (3.1), per-region cap);
seed interior sites on a graded lattice, clearance $\geq h$ from $\mathcal{S}$;
repeat
 unconstrained Delaunay of ($\text{interior} \cup V(\mathcal{S})$);
 relax interior sites by the ODT update (3.8) (surface vertices fixed);
 insert a site at the longest edge of any tetrahedron coarser than the field;
until layout settled **and** no insertion, **or** slivers plateaued;
foreach region r **do** MESHREGION($\mathcal{S}, r, \text{interior}$) // algorithm 9: constrained,
 straddler-free;
assign per-tet region tags; quality + sliver post-pass (carrier-constrained);
return tetrahedra, per-tet region, boundary faces ($= \mathcal{S}$);

8 The quality endgame: perturbation and sliver exudation

Delaunay refinement bounds the radius–edge ratio, and the quality-monotone cleanup of section 7 then drives the worst dihedral upward — but both leave a residual population of *slivers*: tetrahedra whose four vertices are nearly coplanar and nearly cocircular, so their volume and worst dihedral vanish while their radius–edge ratio stays unremarkable. A census of the survivors shows why the monotone regime cannot lift them: their faces fail the 2–3 pierce condition (the opposing apexes’ segment misses the shared face), their edge rings are near-degenerate for the removal DP, and every candidate replacement is no better than the incumbent, so each quality gate rejects the move. Two end-stage operators that do *not* argue via local quality gain resolve this.

8.1 Degeneracy perturbation

Exactly coplanar vertex configurations (the on-carrier chords and caps of curved shells) make the combinatorial operators *undefined*: an exactly zero orientation determinant means no 2–3 flip exists and the ring DP sees only zero-volume candidates. The perturber nudges one movable (interior, unconstrained) vertex of such a tetrahedron by

$$\mathbf{x}' = \mathbf{x} \pm \delta \hat{\mathbf{n}}_{\max}, \quad \delta = 0.05 \ell_{\min}, \quad (8.1)$$

along the dominant facet normal $\hat{\mathbf{n}}_{\max}$ of the flat tetrahedron, gated only on star validity (every incident tetrahedron that was positively oriented stays positively oriented). Crucially, the perturber must not run concurrently with the positional optimizers: a shell’s coplanar configuration *is* the smoother’s local energy optimum, so any mid-loop nudge is smoothed straight back (measured: 16,000 nudges with zero effect on the degeneracy census). It therefore runs only in phases from which smoothing is excluded.

8.2 Sliver exudation on weighted Delaunay triangulations

Exudation [16], [17] assigns each vertex $\mathbf{p}$ a weight $w_p \geq 0$ and replaces the empty-circumsphere criterion by *regularity* with respect to the power distance

$$\pi_{(\mathbf{p}, w_p)}(\mathbf{x}) = \|\mathbf{x} - \mathbf{p}\|^2 - w_p. \quad (8.2)$$

A tetrahedron $(\mathbf{a}, \mathbf{b}, \mathbf{c}, \mathbf{d})$ with weights $(w_a, \dots, w_d)$ has a unique *orthosphere* (the sphere with equal power to all four weighted vertices), and a facet shared with a witness vertex $(\mathbf{e}, w_e)$ is

locally regular iff the witness has positive power to it. The exact predicate is the sign of a 4×4 determinant in Shewchuk difference form, with the weight correction entering only the lifted coordinate:

$$\text{power}(\mathbf{a}, \dots, \mathbf{e}) = \text{sign det} \begin{pmatrix} \mathbf{a} - \mathbf{e} & \|\mathbf{a} - \mathbf{e}\|^2 - (w_a - w_e) \\ \mathbf{b} - \mathbf{e} & \|\mathbf{b} - \mathbf{e}\|^2 - (w_b - w_e) \\ \mathbf{c} - \mathbf{e} & \|\mathbf{c} - \mathbf{e}\|^2 - (w_c - w_e) \\ \mathbf{d} - \mathbf{e} & \|\mathbf{d} - \mathbf{e}\|^2 - (w_d - w_e) \end{pmatrix}, \quad (8.3)$$

evaluated with an interval filter and exact expansion arithmetic as the final word; with all weights zero it degenerates to the classical INSPHERE. *Pumping* a sliver vertex’s weight moves the orthospheres of its star until some facet turns non-regular, and the regular flip that restores regularity removes the sliver — with the full embedding guarantees of a regular triangulation, which is exactly what direct (unguarded) combinatorial surgery cannot offer. Weights are capped by Edelsbrunner’s admissibility bound

$$w_{\mathbf{p}} \leq (0.4 N(\mathbf{p}))^2, \quad N(\mathbf{p}) = \text{distance to the nearest neighbour of } \mathbf{p}, \quad (8.4)$$

which keeps every vertex non-redundant and the weighted triangulation close to the unweighted one. Termination rests on the acyclicity of regular flipping under a *fixed* weight vector (the lifted-volume argument), so each vertex is pumped at most once per optimization call and weighted candidates are decided by (8.3) *alone* — in both directions; mixing quality gates back into a weighted decision forfeits the termination argument.

Three properties of this machinery were established *empirically* on the corpus and shape the implementation:

1. **Exudation needs a Delaunay substrate.** Run as a terminal phase after the quality optimizer — CGAL’s ordering, whose optimizers re-triangulate and therefore preserve Delaunayhood — the power gate degenerates into plain Delaunay drift on our non-Delaunay optimized meshes: “non-regular” holds almost everywhere, and the resulting flips shred the quality structure (capsule: 539 $\rightarrow$ 1619 slivers). The exudation phase therefore runs *first*, on the freshly extracted triangulation, which is still (up to rounding) the Delaunay triangulation of its points; its weights are zeroed before the ordinary optimizer passes begin.
2. **Only true regular flips are acyclic.** The 2–3 flip and the ring-3 edge removal each choose between exactly two configurations, so the power-gated move *is* the regular flip. Releasing the general k -ring removal (whose Klinksek DP picks the max–min-quality triangulation, not the regular one) by the same gate diverges: per-pass removals *grew* 758 $\rightarrow$ 6082 over the phase and watertightness broke.
3. **An anti-degeneracy floor is a manifold/sliver trade.** Power-gated flips must not manufacture near-zero-volume replacements (downstream surgery cannot digest them), so every replacement tetrahedron must beat a one-degree worst dihedral. The floor’s strictness was measured at three settings; one degree maximises watertight count at the cost of the deepest wedge repairs, and manifoldness is the contract metric.

The phase plus the perturber lifted the corpus watertight count by ten geometries in a single run — the largest single improvement in the project’s history — with the sliver census neutral elsewhere.

9 The sizing field, end to end

The single field h threads all stages and is only ever *refined* downward in size:

1. **Edges** set h from curve deflection (5.9) and $h_{\max}$ / per-region overrides.

2. **Faces** take the min of the edge field and the surface deflection bound (6.3); the smoothed result drives the surface CVT.
3. **Volume** carries the face field into the interior from the surface vertices as sources, capped per region, then gradation-limits (3.1) through the interior.

Because each stage only ever lowers h and then re-smooths, the field is monotone and Lipschitz, and the element-size contract (a documented $1.5 \times$ max-edge bound, as in gmsh) holds globally.

9.1 Error control: two orthogonal axes

The user controls the mesh through two *independent* kinds of bound, which must not be conflated.

Conformity (exact). The meshed region fills exactly the volume enclosed by the frozen surface mesh: there are no straddling tetrahedra (proposition 7.1). This is not a tunable tolerance; it is a structural guarantee, verified to machine-rational equality on planar fixtures (section 15). It says nothing about how well the surface mesh approximates a *round* geometry.

Geometric accuracy (a relative tolerance). How closely the piecewise-linear surface follows the true analytic geometry, governed by the deflection bounds $\varepsilon_{\text{edge}} = \text{tol_edge}$ and $\varepsilon_{\text{surf}} = \text{tol_surf}$. By (5.8)/(6.3) an entity of principal radius R is sized $h = R\sqrt{8\varepsilon}$, so the chord deviates by at most εR (*relative*, hence scale-invariant). The default $\varepsilon = 10^{-2}$ (1%) is a good engineering mesh; 10^{-4} is very fine. A round surface is never meshed exactly, so a conformity demand of 10^{-9} on a curved body is a category error: it is the *fill* that is exact, not the chord.

Size and tolerance handles. Element size is capped per dimension by `maxh_edge`, `maxh_surf`, `maxh_vol`, each combined with the global `maxh` as the minimum; there is no volume *tolerance* (the volume size follows from the surface, section 7). Every bound is settable per individual entity through a *hierarchical* handle that mirrors the dimensional hierarchy (Principle 1.4):

```
g.region(sel).surf(sel).edge(sel).maxh  /  .tol
```

A level addressed without a selector applies to all of its children (`g.edge().tol` sets every edge, i.e. the global `tol_edge`; `g.maxh` sets everything), and a selector (`tag`, `normal`, `near`, `between`) narrows it to specific entities. Resolution is most-specific-wins: a per-entity override beats the per-dimension default, which beats the global. The selectors are resolved against the B-rep topology (region $\rightarrow$ faces $\rightarrow$ edges, section 2), so the same composition that structures the algorithm structures its control surface; power users drive local refinement (a boundary layer on one face, a fine seam on one edge) without a separate selection language.

10 Complexity and parallel scaling

Let N be the output vertex count. Every stage is dominated by an incremental Delaunay or a fast-marching sweep, each $O(N \log N)$ with exact predicates: the gradation field (3.2) by heap fast-marching (algorithm 3), the surface/volume triangulations by Bowyer–Watson with octree-accelerated point location. The variational relaxation is a fixed number of passes, each a rebuilt Delaunay plus a linear sweep (the centroid move (3.7) on edges and faces, the ODT move (3.8) in the volume); because the sites are already near-uniform and the per-region constrained fill (algorithm 9) carries no facet-recovery search, the dominant cost is the constrained Delaunay itself, and slivers left for the cleanup pass are rare [9], [15].

The structure is what makes it scale. Within a level the entities are independent (Principle 1.4), so with P workers the wall-clock of a stage is the *slowest single* edge / face / region, not their sum; the only global synchronisations are the two barriers between the three stages.

Multi-region and boolean-heavy scenes parallelise for free over regions, since each region’s constrained tetrahedralization is independent and shares only read-only interface facets. All relaxation queries are local (octree neighbours), so memory and the per-pass cost stay $O(N)$.

11 Why this guarantees conformity

The natural alternative (a single *unconstrained* Delaunay over all points, with each tetrahedron labelled by the region of its centroid) recovers the boundary only *statistically*: where the surface sample is coarser than a feature, a tetrahedron straddles the true surface, and whole-tet labelling then produces a protrusion (centroid inside) or a gap (centroid outside). The algorithm specified here removes that failure mode at its root by (a) completing a watertight, error-controlled surface mesh *first*, and (b) making that mesh a *constraint* on the volume rather than a target to be rediscovered. Conformity is then a theorem (proposition 7.1), not a sampling-density gamble. The cost is the constrained Delaunay / facet recovery, which is exactly the machinery the method must keep; the variational (CVT) relaxation supplies the well-shaped interior that keeps recovery cheap and slivers rare [9], [15].

12 Surface meshing and adaptivity

The same machinery also runs as a *standalone surface mesher*: the planar surface stage (section 6) exported on its own, with no volume, as the 2D product an electromagnetic surface solver (method-of-moments, 2D FEM) consumes. Here quality is the deliverable rather than a by-product of the volume constraint, so the planar patches are triangulated by a guaranteed-quality Delaunay refinement in place of the fixed-pass CVT.

12.1 Sliver-free Ruppert/Chew refinement

Each planar patch is refined by a sizing-field-driven Ruppert/Chew loop [18], [19]: a triangle whose circumradius-to-shortest-edge ratio exceeds the bound (equivalently, whose smallest angle is below the target) has its circumcenter inserted; a boundary segment encroached by such an insertion is split at its midpoint instead. The loop terminates with a guaranteed *minimum angle* — the implementation targets 20° — so the output is *sliver-free at any density*, and no triangle exceeds the local sizing target $h(\mathbf{x})$, so the mesh grades smoothly. An apex-indexed encroachment test and a hash-grid spacing guard keep the refinement near-linear (the two quadratic costs of a naive Ruppert removed).

A *boundary-preserving* optimal-Delaunay (ODT) smoothing then equalises the interior: each free vertex moves toward the area-weighted ODT target (3.8), under-relaxed, with the boundary frozen and a per-pass guard that reverts any pass which would drop the global minimum angle below the bound. This keeps the Ruppert guarantee while removing the density jump a bare refinement leaves just inside the boundary.

12.2 Element and triangle budgets

Both meshers take an optional *budget*. `mesh(target_elements=N)` retunes the single global size scale over a few remeshes: because the element count scales as scale^{-3} , each step multiplies the scale by $(n/N)^{1/3}$, so the final count lands within a few percent of N while the *relative* refinement (curvature, size points) keeps its shape. `surface_mesh(target_triangles=N)` is instead a *cap*: the count-driven Ruppert resolves the field down to its floor $h_{\min}$ but stops once the triangle count reaches N (the budget split across patches by area), spending its last splits on the worst-quality triangles. One mesh therefore yields $\min(\text{field-resolved}, \text{budget})$, still graded and sliver-free.

12.3 Adaptive refinement (Dörfler marking)

The surface mesher closes the MARK $\rightarrow$ REFINES half of an adaptive loop; SOLVE and ESTIMATE belong to the solver. Given a per-triangle error indicator η_t (an a-posteriori estimator, e.g. $\text{area} \cdot |\nabla u|$), *Dörfler bulk marking* [20] selects the smallest element set $\mathcal{M}$ whose summed *squared* indicator reaches a fraction θ of the total,

$$\sum_{t \in \mathcal{M}} \eta_t^2 \geq \theta \sum_t \eta_t^2, \quad \theta \in (0, 1] \text{ (default 0.5)}, \quad (12.1)$$

the energy-norm convention. Each marked element becomes a *point size source* at h_{local} /factor (clamped to h_{min}): the mesher’s gradient-limited grading spreads it into a smooth background field and the Ruppert refinement re-triangulates, so *every adapted mesh stays sliver-free* with the same guaranteed minimum angle. One ESTIMATE $\rightarrow$ MARK $\rightarrow$ REFINES step is `refine_dorfler`; iterated, it concentrates resolution where the error lives.

The indicator must be a *proper* a-posteriori estimator — the per-element error, which *shrinks* as an element refines ($\eta \sim h^k$). Then $\mathcal{M}$ stays small and the loop converges. A pointwise field that does *not* shrink on refinement (a bare e^{-r^2} , say) makes Dörfler re-mark the same region every pass, so the refinement never localises — it “refines everywhere”, the failure the squared-indicator marking exists to avoid.

13 Discrete carriers: remeshing imported triangle soups

STL and OBJ imports enter the pipeline as first-class *discrete carriers* rather than as frozen facets. Freezing an imported soup facet by facet turns every input edge into a protected feature: the import is then pinned, its slivers are permanent, and no sizing decision can be revisited. The discrete carrier instead treats the soup as the *envelope* of an unknown smooth surface: the mesher remeshes against it through the same three-call oracle as every analytic carrier (section 7.5), and the input tessellation survives only as geometry, never as combinatorics.

13.1 Crease segmentation and validation

On import, exactly degenerate facets are dropped, vertices are welded by exact bit-match, and facets are grouped into smooth regions by a union-find that merges across an edge shared by facets i, j iff their unit normals satisfy

$$\langle \hat{\mathbf{n}}_i, \hat{\mathbf{n}}_j \rangle \geq \cos \theta_c, \quad \theta_c = 40^\circ \text{ by default}, \quad (13.1)$$

so region boundaries are exactly the crease edges; each region becomes one carrier (an AABB-tree accelerated closest-point structure), and the crease curves feed the ordinary feature-edge machinery (protecting balls, curve distribution). A solid import must pass a watertightness check — every edge shared by exactly two facets with opposite orientation — before it is accepted; classics that fail (holes, duplicate facets) are parked for mesh repair rather than silently mis-meshed.

13.2 Curvature estimation and its two governors

The carrier estimates a per-facet curvature radius from the normal turn across each interior edge: for facets i, j with centroid distance d_{ij} and normal angle θ_{ij} , the osculating-circle chord approximation gives

$$r_i = \min_{j \sim i} \frac{d_{ij}}{\theta_{ij}}, \quad (13.2)$$

which feeds the same sagitta-bounded sizing $h_{\text{curv}} = r\sqrt{8\text{tol}}$ as the analytic carriers. Unlike analytic curvature, (13.2) is an *estimate* on a noisy, often over-tessellated input, and unbounded

it overrides the user’s size target wholesale (measured: $\times 39$ tetrahedra on one scan, $\times 14$ on another). Two governors bound its authority. First, curvature below the input tessellation’s own resolution is normal noise, not information:

$$r_i^{\text{eff}} = \max(r_i, 4\ell_i^{\text{max}}), \quad (13.3)$$

with ℓ_i^{max} the facet’s longest edge. Second, the estimated curvature may undercut the local user target h_{base} by at most a fixed factor,

$$h = \max(h_{\text{curv}}, \tfrac{1}{4} h_{\text{base}}), \quad (13.4)$$

while exact analytic curvature keeps its full authority. Both governors were sized on the corpus: without (13.3) flat-noise regions refine to the input edge length; without (13.4) finely tessellated scans refine $\sim 7\times$ below maxh and the volume explodes cubically.

13.3 Analytic crossings on closed patches

The crossing oracle on a discrete patch follows the candidate–verify pattern of section 7.5: one walk of the patch’s AABB tree with segment-slab pruning collects Möller–Trumbore [21] candidates — for triangle $(\mathbf{p}_0, \mathbf{p}_1, \mathbf{p}_2)$ and segment direction $\mathbf{d} = \mathbf{b} - \mathbf{a}$,

$$\begin{pmatrix} t \\ u \\ v \end{pmatrix} = \frac{1}{\langle \mathbf{e}_1, \mathbf{d} \times \mathbf{e}_2 \rangle} \begin{pmatrix} \langle \mathbf{e}_2, (\mathbf{a} - \mathbf{p}_0) \times \mathbf{e}_1 \rangle \\ \langle \mathbf{a} - \mathbf{p}_0, \mathbf{d} \times \mathbf{e}_2 \rangle \\ \langle \mathbf{d}, (\mathbf{a} - \mathbf{p}_0) \times \mathbf{e}_1 \rangle \end{pmatrix}, \quad \mathbf{e}_k = \mathbf{p}_k - \mathbf{p}_0, \quad (13.5)$$

kept for $t \in (0, 1)$, $u, v \geq 0$, $u + v \leq 1$ — and sign probes of (7.2) at $t \pm \delta$ decide transversality, so edge grazes and the double counts of shared-edge hits fall out (the offset sign does not flip there). The probes must sit *next to* the candidates, not at far partition midpoints: an open patch’s offset field beyond its rim projects onto the rim and is not sign-faithful, and midpoint probing there dropped real crossings wholesale. For the same reason the analytic arm is gated to *closed* patches (no rim edge): there facet hits and field crossings agree, the no-candidate common case costs no projection at all, and end-to-end meshing of a closed scan accelerated $\sim 8\times$; open (crease-bounded) patches remain on the sphere trace (7.3), whose dense samples are rim-safe.

13.4 Bridge faces: a fourth defect class

Imported and boolean geometry exposed a defect the vertex-based straddler test is blind to: a boundary face whose vertices all sit on surfaces while its *interior* spans a cavity opening — a lid, topologically watertight yet geometrically false. The detector classifies a face with longest edge ℓ , vertex off-surface distances $\{o_k\}$ and centroid off-surface distance o_c as a *bridge face* iff

$$o_c > \tfrac{1}{4} \ell \quad \text{and} \quad \max_k o_k \leq \tfrac{1}{4} \ell, \quad (13.6)$$

i.e. every vertex passes the straddler test but the face interior does not. Bridge faces are counted, located, and rendered as a separate marker class in the debug gallery; on first measurement the class concentrated on one mold geometry (131 faces over its cavity openings) with a long tail across organic imports — previously invisible under every existing metric except a raised maximum surface deviation.

14 FEM and MoM export

Because the mesher is a pure-Rust library, its meshes are consumed in-process by sibling Rust solvers (the method-of-moments code `rapidmom` bundles `rapidmesh-tet` as a crate). Beyond the raw points and triangles, the surface mesh exposes the derived connectivity a surface solver would otherwise rebuild (`rapidmesh-tet::mom`):

- *edge adjacency* — each undirected edge mapped to its up-to-two incident triangles and their conductor tags;
- *RWG basis edges* — the surface degrees of freedom: interior edges shared by two triangles of the *same* conductor, returned as $[v_0, v_1, t_+, t_-]$ with the current orientation $+\rightarrow-$;
- the *conductor outline* (boundary edges — a free side or a tag change) and *port edges* (boundary edges on a query line), the excitation the solver feeds;
- per-triangle *area* and *minimum angle*, the element-quality field (all $\geq$ the Ruppert bound).

These are computed natively and validated bit-identical to a reference NumPy implementation, so a Rust solver drives the whole mesh $\rightarrow$ assemble pipeline with no Python in the loop.

Hierarchical per-entity sizing. The geometry API exposes a fluent selector, `g.region(s).surf(s).edge(s)` that resolves a scope to B-rep entity ids natively (`rapidmesh_brep::Topology`): a region by tag, a face by tag/normal/proximity, an edge by its bounding regions, kind, or proximity. A bare dimension scope (`g.surf().maxh`) sets the global per-dimension target; a filtered scope sets the per-entity overrides the sizing field then honours, so a single curved face or one conductor edge is refined without touching the rest.

15 Validation

The algorithm is validated on a from-scratch corpus of 102 geometries spanning ten families: *2D plates* (the planar surface stage in isolation), *analytic primitives*, *CSG booleans* and multi-region assemblies, hierarchical *sizing* and *gradation* cases, parametric *shape* sweeps, *showcase* models, *RF/EM* structures, *passives* (method-of-moments spiral inductors and transformers, meshed as surfaces), and — new — *imports*: the standard surface-mesh reference models the field benchmarks against (fandisk, spot, homer, cheburashka, armadillo, ...), fetched as OBJ, validated watertight, and remeshed through the discrete-carrier path of section 13. Each geometry is meshed at its default density and rendered headlessly by the project’s WebGPU rasterizer — which runs the *same* scene pipeline as the live viewer, with no browser — into a NORMAL view (region-coloured, volumes clipped to expose the interior) and a DEBUG view that overlays the located defect markers (sliver, straddler, non-manifold edge, and the bridge faces of (13.6)) and the per-stage meshing time. The full per-geometry metric set (point and element counts, minimum dihedral angle, watertightness, sliver / straddler / bridge / non-manifold-edge counts, maximum surface deviation) is tabulated per family in the longtables below, and every mesh is shown in the gallery at the end of this section.

Discipline. Every run is date-versioned into `bench/history/` with its git metadata, and the runner prints the headline trajectory against the previous run, so regressions surface at the run that caused them. A hard per-geometry *runtime budget of five minutes* keeps the corpus practicable; reference models that blow it (the 100k-facet scan busts, and one scan whose noise fragments into 66 crease regions) are parked, each with its measured cause, until the speed work brings them under budget. Sub-corpus runs (`--sub family`) and a curated quick tier (`--quick`) serve as fast mid-iteration gates without writing history; the full run remains the single benchmark truth.

Summary. 102 of 102 geometries mesh. Of the 81 volume meshes, 72 are watertight, 77 are straddler-free and 57 are sliver-free; the analytic-primitive, boolean and box-derived (sizing, gradation) families are watertight at the expected isotropic quality (minimum dihedral typically $12-25^\circ$), and the endgame of section 8 lifted the watertight count by ten geometries in its first full run. The residual failures concentrate on *curved material interfaces* — where a chosen curved-facet triangulation is not force-recovered (section 7) — on dense multi-region assemblies, on the on-carrier sliver bands of thin curved shells (the pressure-vessel class), and on the cavity-lid bridge faces that (13.6) now makes countable. These are the standing open items the corpus

exists to track. Conformity on planar and single-curved geometry is exact by proposition 7.1 and independent of density: the boundary-constrained, per-region tetrahedralization (algorithm 9) makes the frozen surface a hard constraint and discards every straddling tetrahedron, so a region’s boundary is exactly the prescribed subset of $\mathcal{S}$. The validation harness (`report/corpus.py`, `report/render_gallery.py`) re-runs the whole matrix and regenerates these tables and figures on demand, so the corpus is a standing regression gate.

Table 3: Surface meshes: 2D plates (planar surface stage).

geometry	points	faces
annulus	1777	3386
disc	1005	1944
hexagon	1021	1956
l_polygon	1025	1932
pentagon	1077	2067
plate_rect	642	1212
square_hole	2353	4498
star	1044	1946
triangle	748	1409

Table 4: Volume meshes: Analytic primitives.

geometry	points	tets	min-dih	wt	sliv	strad	bridge	non-mf	surf-dev
box	319	1218	24.1°	✓	0	0	0	0	0.00e+00
cone	350	1382	20.2°	✓	0	0	0	0	6.57e-02
cylinder	513	2217	21.6°	✓	0	0	0	0	1.49e-02
ellipsoidish	2350	11907	4.1°	✓	1	0	0	0	5.76e-03
frustum	483	2019	4.3°	✓	3	0	0	0	1.80e-02
hex_prism	1550	7303	26.0°	✓	0	0	0	0	0.00e+00
prism_l	1231	5314	25.0°	✓	0	0	0	0	0.00e+00
slab	741	2861	24.6°	✓	0	0	0	0	0.00e+00
sphere	862	4021	28.0°	✓	0	0	0	0	1.51e-02
star_prism	1610	6874	24.5°	✓	0	0	0	0	0.00e+00
torus	2536	11246	5.0°	✓	3	0	0	0	1.77e-02
tri_prism	1027	4579	23.2°	✓	0	0	0	0	0.00e+00
wedge	821	3443	24.3°	✓	0	0	0	0	0.00e+00

Table 5: Volume meshes: CSG booleans and multi-region assemblies.

geometry	points	tets	min-dih	wt	sliv	strad	bridge	non-mf	surf-dev
box_minus_2sph	1621	6371	24.2°	✓	0	0	0	0	1.35e-02
capsule	6507	34432	0.0°	✓	535	1	0	0	1.84e-02
cross_cyl	1668	7367	3.0°	✓	1	0	0	0	1.42e-02
diff_box_cyl	1903	8452	21.5°	✓	0	0	0	0	1.02e-02
diff_box_sphere	1660	7445	27.2°	✓	0	0	0	0	1.07e-02
diff_cyl_box	1081	4451	0.2°	✓	38	0	0	0	8.51e-03
drilled_block	3016	13747	21.1°	✓	0	0	0	0	9.75e-03
flush_tee	445	1592	21.6°	✓	0	0	0	0	0.00e+00

geometry	points	tets	min-dih	wt	sliv	strad	bridge	non-mf	surf-dev
fused_deep	2399	12152	24.5°	✓	0	0	0	0	8.97e-03
fused_three	2630	13170	26.6°	✓	0	0	0	0	1.01e-02
fused_two	1806	8931	23.8°	✓	0	0	0	0	1.10e-02
fused_unequal	1890	9194	20.7°	✓	0	0	0	0	1.29e-02
nested_spheres	2601	13114	24.0°	✓	0	0	0	0	1.10e-02
sphere_minus_box	1655	7208	26.4°	✓	0	0	0	0	7.21e-03
tube	1679	7348	3.4°	✓	1	0	0	0	1.07e-02
union_box_cyl	1426	6347	25.7°	✓	0	0	0	0	7.81e-03
union_box_sphere	1570	7354	24.7°	✓	0	0	0	0	1.13e-02
via	2943	14801	21.0°	✓	0	0	0	0	5.93e-03

Table 6: Volume meshes: Imported reference models (discrete carriers).

geometry	points	tets	min-dih	wt	sliv	strad	bridge	non-mf	surf-dev
armadillo	9824	45753	0.0°	—	3	0	27	13	1.67e+00
cheburashka	4427	19933	0.0°	✓	12	0	4	0	1.09e-02
cow	13508	69281	0.0°	—	35	0	11	11	8.33e-02
fandisk	4349	20901	7.2°	✓	1	0	0	0	5.06e-02
homer	7877	36617	13.0°	✓	0	0	12	0	8.29e-03
spot	5549	27873	7.8°	✓	3	0	2	0	1.61e-02

Table 7: Volume meshes: Showcase models.

geometry	points	tets	min-dih	wt	sliv	strad	bridge	non-mf	surf-dev
baffled_tank	2862	14378	23.2°	✓	0	0	0	0	0.00e+00
bearing	8211	35664	18.8°	✓	0	0	0	0	5.32e-03
bracket	6768	30257	21.9°	✓	0	0	0	0	5.37e-03
chain	13165	58035	13.2°	✓	0	0	0	0	1.32e-02
coax_step	7489	34911	0.0°	✓	357	0	0	0	1.31e-04
counterbore_plate	6937	32771	20.1°	✓	0	0	0	0	5.78e-03
dice	7630	38113	20.0°	✓	0	0	0	0	1.36e-02
gear	1921	6747	0.0°	✓	1	0	0	0	0.00e+00
house	3795	15680	16.3°	✓	0	0	0	0	4.30e-03
lattice_cube	8332	39491	20.8°	✓	0	0	0	0	1.46e-02
microstrip	13362	71609	20.0°	✓	0	0	0	0	0.00e+00
mold_block	17956	98442	10.5°	✓	0	0	131	0	1.87e-01
nested_shells	4174	16186	20.9°	✓	0	0	0	0	1.14e-02
orbs	2531	11787	0.0°	✓	52	2	0	0	1.49e-02
perforated_plate	17693	76876	20.8°	✓	0	0	0	0	8.60e-03
pipe_cross	7760	31517	11.8°	—	0	0	0	2	1.85e-02
pipe_junction	4381	18010	7.5°	—	2	0	0	2	1.35e-02
pressure_vessel	8583	35136	0.0°	—	3226	0	2	13	8.05e-02
rocket	3155	14120	5.6°	✓	2	0	0	0	2.45e-02
serpentine	9150	39252	0.1°	—	31	3	0	7	1.93e-02
sizing_field	1796	9338	20.9°	✓	0	0	0	0	0.00e+00
spring	30275	131184	0.1°	—	115	0	3	137	9.20e-02

geometry	points	tets	min-dih	wt	sliv	strad	bridge	non-mf	surf-dev
square_to_round	1664	4893	0.0°	—	160	0	0	12	0.00e+00

Table 8: Volume meshes: RF/EM structures.

geometry	points	tets	min-dih	wt	sliv	strad	bridge	non-mf	surf-dev
rf_coax_step	6427	36523	0.0°	—	18	0	0	3	6.71e-04
rf_cored_inductor	50465	298287	20.1°	✓	0	3	0	0	5.80e-05
rf_dielectric_resonator	9509	55184	21.8°	✓	0	0	0	0	1.50e-04
rf_iris_filter	4283	21237	20.7°	✓	0	0	0	0	0.00e+00
rf_microstrip_line	3056	14224	20.8°	✓	0	0	0	0	0.00e+00
rf_patch_antenna	5132	25877	21.3°	✓	0	0	0	0	0.00e+00
rf_solenoid	42815	252727	20.1°	✓	0	0	0	0	4.70e-05
rf_toroid_core	8077	46073	20.9°	✓	0	0	0	0	5.30e-05
rf_transformer	74424	439353	18.3°	✓	0	0	0	0	4.60e-05

Table 9: Volume meshes: Sizing.

geometry	points	tets	min-dih	wt	sliv	strad	bridge	non-mf	surf-dev
box_coarse_interior	1643	7142	23.8°	✓	0	0	0	0	0.00e+00
box_linear	224	849	22.1°	✓	0	0	0	0	0.00e+00
sphere_coarse_interior	1718	7738	21.8°	✓	0	0	0	0	7.36e-03

Table 10: Surface meshes: Sizing.

geometry	points	faces
blob_graded	1161	2142
disc_coarse_interior	457	832
gear_coarse_interior	343	600
plate_coarse_interior	332	610
plate_linear	373	694

Table 11: Surface meshes: Shape.

geometry	points	faces
gear_plate	4262	8242
interdigital_comb	2258	4166
organic_blob	4230	8284
spiral_inductor	1985	3330

Table 12: Volume meshes: Graded.

geometry	points	tets	min-dih	wt	sliv	strad	bridge	non-mf	surf-dev
box_core_fine	1186	6356	25.3°	✓	0	0	0	0	0.00e+00
box_edge_fine	1162	5481	23.8°	✓	0	0	0	0	0.00e+00
box_face_fine	1371	6258	25.2°	✓	0	0	0	0	0.00e+00
cyl_coarse_interior	1174	5352	20.1°	✓	0	0	0	0	3.47e-02
notch_edge_fine	1033	5198	22.9°	✓	0	0	0	0	0.00e+00
sphere_patch_fine	2050	11077	25.9°	✓	0	0	0	0	2.20e-02
stacked_two_region	3646	18525	23.5°	✓	0	0	0	0	0.00e+00

Table 13: Volume meshes: Passives.

geometry	points	tets	min-dih	wt	sliv	strad	bridge	non-mf	surf-dev
rp_spiral_fem	5994	29365	9.4°	✓	1	0	0	0	0.00e+00
rp_sym_inductor_fem	4972	23793	8.9°	✓	3	0	0	0	0.00e+00

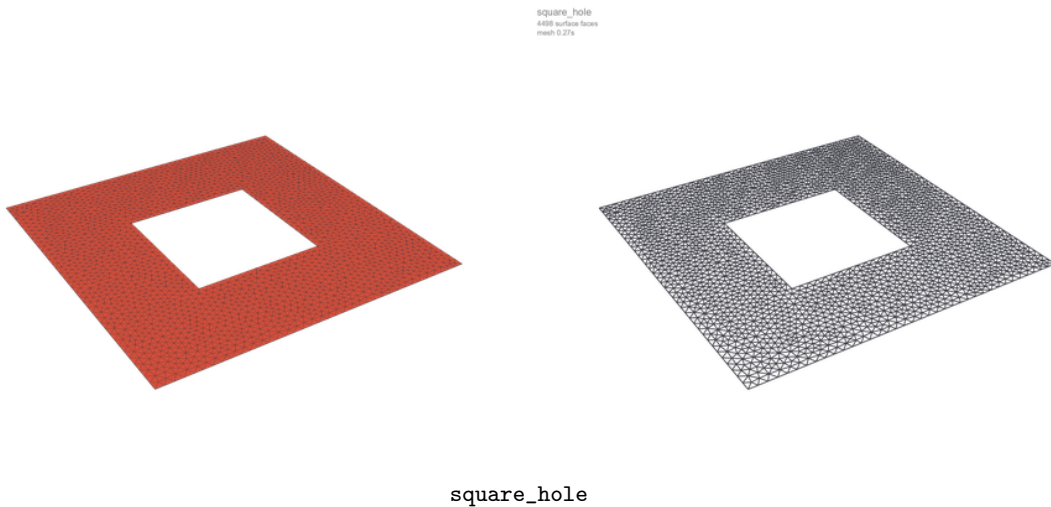
Table 14: Surface meshes: Passives.

geometry	points	faces
rp_spiral_mom	3113	5458
rp_sym_inductor_mom	2093	3546
rp_sym_transformer_mom	3466	5782

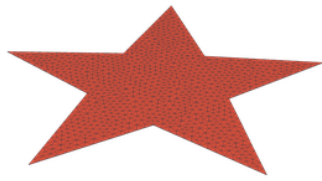
15.1 Gallery

A representative geometry or two from each family; the NORMAL view (left: region-coloured solid, volumes clipped in y to expose the interior tetrahedra) sits beside the DEBUG view (right: surface wireframe, located defect markers, and a metrics + meshing-time overlay). The full per-geometry metric set for the entire corpus is in the longtables above.

2D plates (planar surface stage)



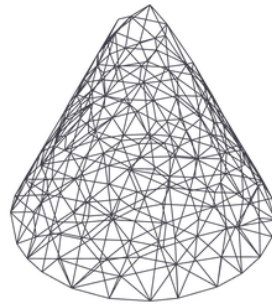
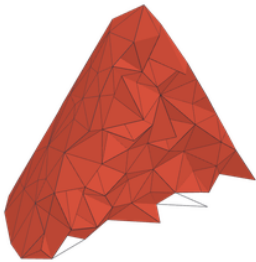
star
1945 surface faces
mesh 0.13s



star

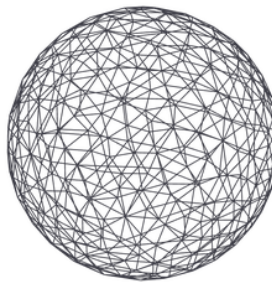
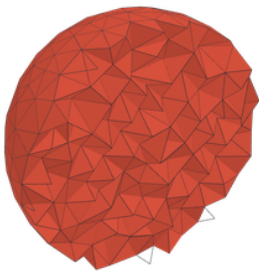
Analytic primitives

cone
1382 tetra min-dih 20.2 deg
watertight yes
silvers 5 straddlers 0 non-manifold 0
max surface dev 0.0637
mesh 2.62s
opt 1.689 ms



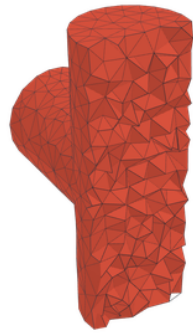
cone

sphere
6521 tetra min-dih 28.0 deg
watertight yes
silvers 5 straddlers 0 non-manifold 0
max surface dev 0.0791
mesh 1.62s
opt 83 ms

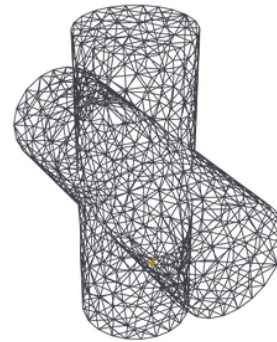


sphere

CSG booleans and multi-region assemblies



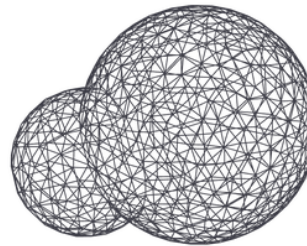
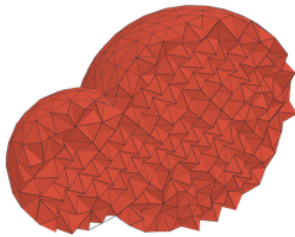
cross_cyl
7357 tets, min-dth 3.0 deg
watertight yes
silvers 5, all-paddlers 0, non-manifold 0
max surface dev 0.0742
mesh 2.27s
opt 417 ms



defects:
■ silver

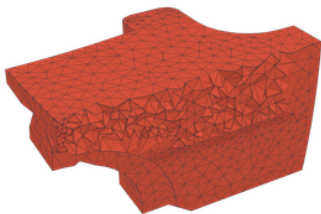
cross_cyl

fused_unequal
9194 tets, min-dth 20.7 deg
watertight yes
silvers 0, all-paddlers 0, non-manifold 0
max surface dev 0.0729
mesh 1.17s
opt 208 ms

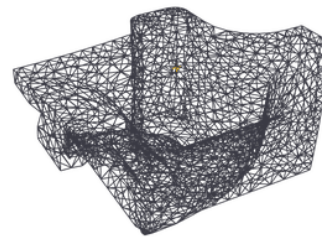


fused_unequal

Imported reference models (discrete carriers)

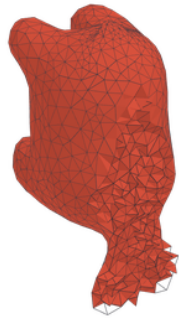


fandisk
20901 tets, min-dth 7.2 deg
watertight yes
silvers 1, all-paddlers 0, non-manifold 0
max surface dev 0.0506
mesh 17.07s
opt 2700 ms

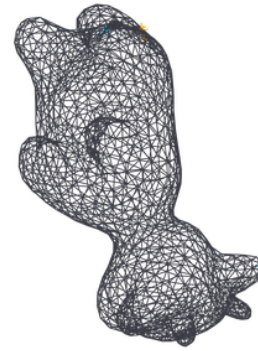


defects:
■ silver

fandisk



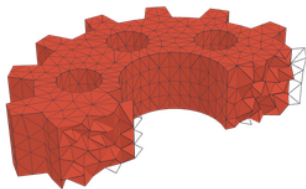
spot
27873 tets min-dth 7.8 deg
watertight yes
silvers 3 straddlers 0 non-manifold 0
max surface dev 0.07191
mesh 9.05s
opt 2911 ms



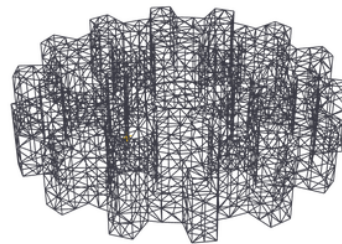
defaults
silvers
bridge_face

spot

Showcase models

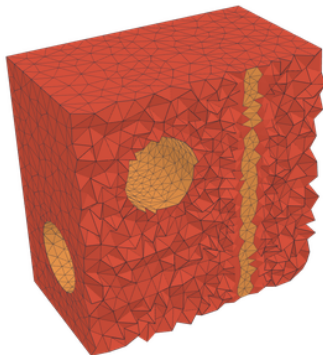


gear
6747 tets min-dth 0.0 deg
watertight yes
silvers 5 straddlers 0 non-manifold 0
max surface dev 0.0000
mesh 1.25s
opt 221 ms

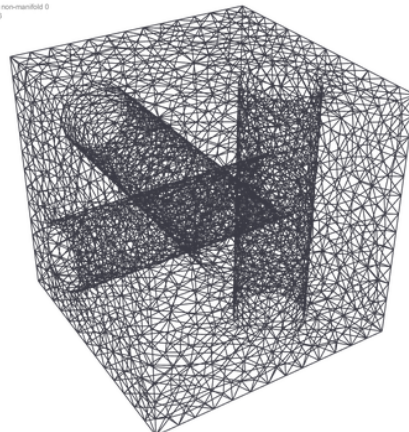


defaults
silvers

gear

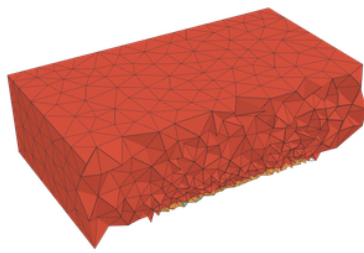


lattice_cube
39491 tets min-dth 20.8 deg
watertight yes
silvers 0 straddlers 0 non-manifold 0
max surface dev 0.07140
mesh 7.26s
opt 1639 ms

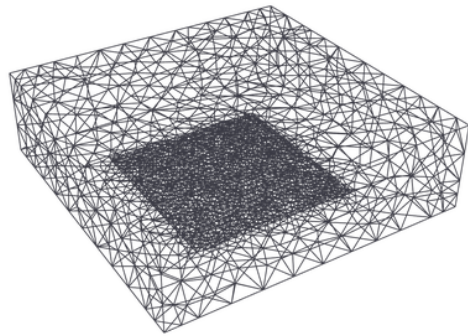


lattice_cube

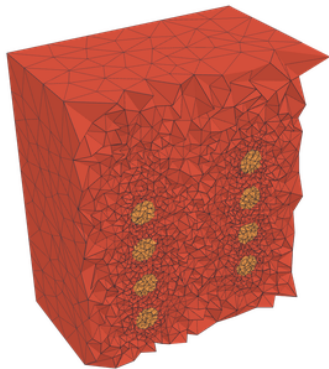
RF/EM structures



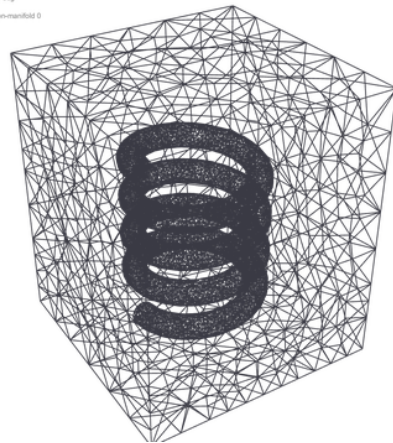
```
rf_patch_antenna
23577 tets, min-dih 21.9 deg
wateright yes
silvers 0, straddlers 0, non-manifold 0
max surface dev 0.0000
mesh 2.36e4
opt 1392 ms
```



rf_patch_antenna

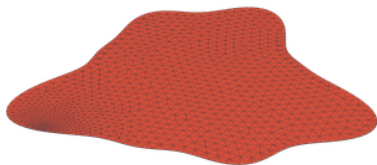


```
rf_solenoid
23577 tets, min-dih 20.1 deg
wateright yes
silvers 0, straddlers 0, non-manifold 0
max surface dev 0.0000
mesh 60.44e4
opt 16558 ms
```

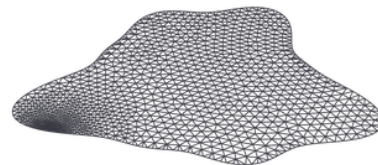


rf_solenoid

Sizing



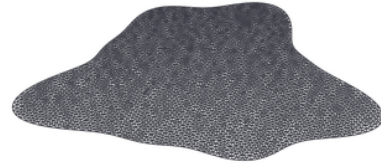
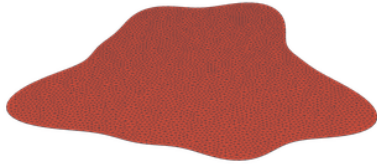
```
blob_graded
2142 surface faces
mesh 1.67e
```



blob_graded

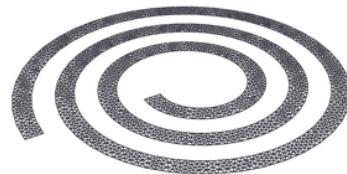
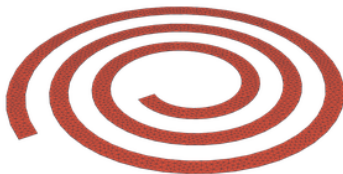
Shape

organic_blob
6284 surface faces
mesh 2.62s



organic_blob

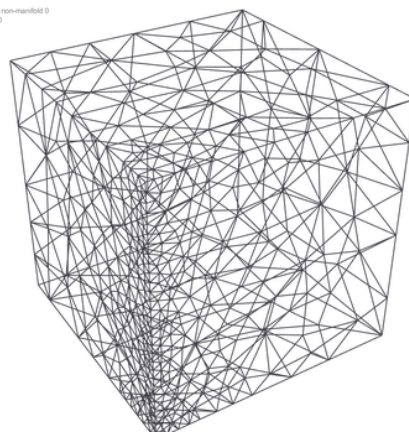
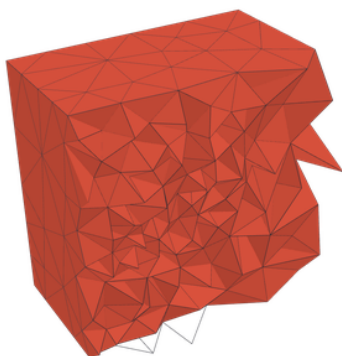
spiral_inductor
3320 surface faces
mesh 1.43s



spiral_inductor

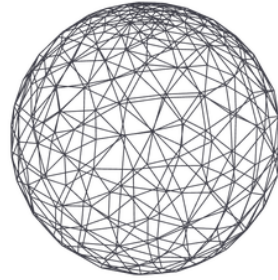
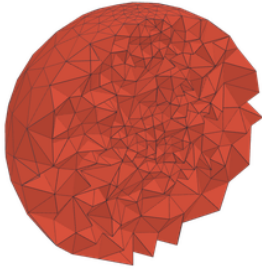
Graded

box_edge_fine
5487 tets - min-dih 23.8 deg
swweight yes
silvers 0 straddlers 0 non-manifold 0
max surface dev 0.0000
mesh 0.97s
opt 696 ms



box_edge_fine

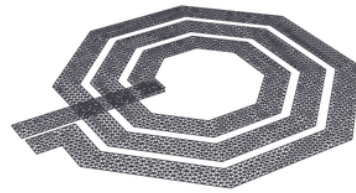
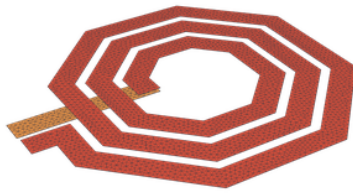
```
sphere_patch_fine
11077 tex  tex-eth 25.9 deg
sawright_00
sigma 0  straddlers 0  non-manifold 0
max surface dev 0.0020
mesh 4.80s
opt 2.958 ms
```



sphere_patch_fine

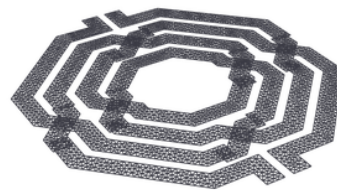
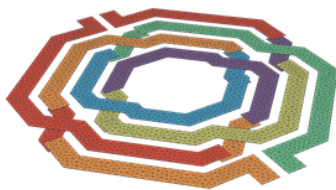
Passives

```
rp_spiral_mom
5458 surface faces
mesh 0.75s
```



rp_spiral_mom

```
rp_sym_transformer_mom
3782 surface faces
mesh 0.36s
```



rp_sym_transformer_mom

References

- [1] J. R. Shewchuk, “Adaptive precision floating-point arithmetic and fast robust geometric predicates,” *Discrete & Computational Geometry*, vol. 18, no. 3, pp. 305–363, 1997.

- [2] H. Borouchaki, F. Hecht, and P. J. Frey, “Mesh gradation control,” *International Journal for Numerical Methods in Engineering*, vol. 43, no. 6, pp. 1143–1165, 1998.
- [3] Q. Du, V. Faber, and M. Gunzburger, “Centroidal Voronoi tessellations: Applications and algorithms,” *SIAM Review*, vol. 41, no. 4, pp. 637–676, 1999.
- [4] S. P. Lloyd, “Least squares quantization in PCM,” *IEEE Transactions on Information Theory*, vol. 28, no. 2, pp. 129–137, 1982.
- [5] Q. Du and D. Wang, “Anisotropic centroidal Voronoi tessellations and their applications,” *SIAM Journal on Scientific Computing*, vol. 26, no. 3, pp. 737–761, 2005.
- [6] B. Lévy and Y. Liu, “ L_p centroidal Voronoi tessellation and its applications,” *ACM Transactions on Graphics (SIGGRAPH)*, vol. 29, no. 4, pp. 1–11, 2010.
- [7] H. Edelsbrunner and N. R. Shah, “Triangulating topological spaces,” *International Journal of Computational Geometry & Applications*, vol. 7, no. 4, pp. 365–378, 1997.
- [8] J.-D. Boissonnat and S. Oudot, “Provably good sampling and meshing of surfaces,” *Graphical Models*, vol. 67, no. 5, pp. 405–451, 2005.
- [9] P. Alliez, D. Cohen-Steiner, M. Yvinec, and M. Desbrun, “Variational tetrahedral meshing,” *ACM Transactions on Graphics (SIGGRAPH)*, vol. 24, no. 3, pp. 617–625, 2005.
- [10] L. Piegl and W. Tiller, *The NURBS Book*, 2nd ed. Springer, 1997.
- [11] D.-M. Yan, B. Lévy, Y. Liu, F. Sun, and W. Wang, “Isotropic remeshing with fast and exact computation of restricted Voronoi diagram,” *Computer Graphics Forum (SGP)*, vol. 28, no. 5, pp. 1445–1454, 2009.
- [12] H. Si, “TetGen, a Delaunay-based quality tetrahedral mesh generator,” *ACM Transactions on Mathematical Software*, vol. 41, no. 2, pp. 1–36, 2015.
- [13] J. R. Shewchuk, “Constrained Delaunay tetrahedralizations and provably good boundary recovery,” in *11th International Meshing Roundtable*, 2002, pp. 193–204.
- [14] S.-W. Cheng, T. K. Dey, and J. R. Shewchuk, *Delaunay Mesh Generation*. CRC Press, 2012.
- [15] J. Tournois, C. Wormser, P. Alliez, and M. Desbrun, “Interleaving Delaunay refinement and optimization for practical isotropic tetrahedron mesh generation,” *ACM Transactions on Graphics (SIGGRAPH)*, vol. 28, no. 3, pp. 1–9, 2009.
- [16] S.-W. Cheng, T. K. Dey, H. Edelsbrunner, M. A. Facello, and S.-H. Teng, “Sliver exudation,” *Journal of the ACM*, vol. 47, no. 5, pp. 883–904, 2000.
- [17] H. Edelsbrunner and D. Guoy, “An experimental study of sliver exudation,” in *Engineering with Computers*, vol. 18, 2002, pp. 229–240.
- [18] J. Ruppert, “A Delaunay refinement algorithm for quality 2-dimensional mesh generation,” *Journal of Algorithms*, vol. 18, no. 3, pp. 548–585, 1995.
- [19] L. P. Chew, “Guaranteed-quality mesh generation for curved surfaces,” in *Proc. 9th Annual Symposium on Computational Geometry (SoCG)*, 1993, pp. 274–280.
- [20] W. Dörfler, “A convergent adaptive algorithm for Poisson’s equation,” *SIAM Journal on Numerical Analysis*, vol. 33, no. 3, pp. 1106–1124, 1996.
- [21] T. Möller and B. Trumbore, “Fast, minimum storage ray-triangle intersection,” *Journal of Graphics Tools*, vol. 2, no. 1, pp. 21–28, 1997.